

UNIFIED MODELLING LANGUAGE COM 213

LECTURE NOTE

System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system. It is about representing a system generally using some kind of graphical notation, which is now almost always based on notations in the **Unified Modeling Language (UML)**. Model is a representation of a system (an idea of the system being studied rather than an alternative representation of that system or a simplification of reality). It helps the analyst to understand the functionality and understand better the system we are developing; they are used to communicate with customers.

Models can be for both new and existing system. Models of the **existing systems** help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system. Models of the **new system** are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.

You might create various models to illustrate the system from different perspectives. For instance

- An External perspective, where you model the context or environment of the system.
- An Interaction perspective, where you model the interactions between a system and its environment, or between the components of a system.
- A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events.

Importance of Modeling

It helps development team and their users visualize the final product.

Models can validate our theories or try out new ones with minimal risk and cost.

We build models of complex systems because we cannot comprehend such a system in its entirety.

Five (5) types of UML diagrams that are the most useful for system modeling:

Activity diagrams, which show the activities involved in a process or in data processing.

Use case diagrams, which show the interactions between a system and its environment.

Sequence diagrams, which show interactions between actors and the system and between system components.

Class diagrams, which show the object classes in the system and the associations between these classes.

State diagrams, which show how the system reacts to internal and external events.

Types of system models

- a. Context models
- b. Interaction models
- c. Structural models
- d. Behavioral models

a. Context models: The context model is used to specify the boundaries of a system. It represents the system as a whole. It is expected that during specification of a system, you need to decide on the system boundaries. This involves working with system stakeholders to decide what functionality should be included in the system and what is provided by the system's environment. Looking at possible overlaps in functionality with existing systems and decide where new functionality should be implemented. These decisions should be made early in the process to limit the system costs and the time needed for understanding the system requirements and design.

System boundaries are established to define what is inside and what is outside the system. They show other systems that are used or depend on the system being developed. The position of the system boundary has a profound effect on the system requirements. Defining a system boundary is a political judgment since there may be pressures to develop system boundaries that increase/decrease the influence or workload of different parts of an organization.

b. Interaction models: All systems involve interaction of some kind. Some types of interactions represented in a model are: User interaction, which involves user inputs and outputs, interaction between the system being developed and other systems or interaction between the components of the system. Modeling user interaction is important as it helps to identify user requirements. Modeling system to system interaction highlights the communication problems that may arise. Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.

USE CASES

A use case describes "who" can do "what" with the system in question. It can also be defined as a sequence of transaction in a system whose task is to yield a measurable value to an individual actor of the system. It represents a discrete task that involves external interaction with a system. The use case technique is used to capture a system's behavioural requirements by detailing scenario-driven threads through the functional requirements. Functional requirements are the features of the system expected to be part of the system after development.

Use Case Diagrams

A UCD has four major elements:

1. The system described

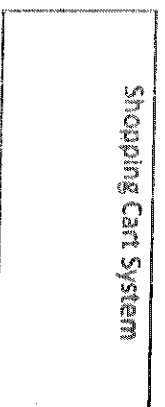
2. The actors that the system interacts with
3. The use cases, or service, that the system knows how to perform
4. The relationships between the above elements

System

As part of use-case modelling, the boundaries of the system developed must be defined. Defining the boundaries of the system is not difficult. The following questions are asked.

- Which tasks are automated and which are manual?
- Which tasks are performed by other systems?

The entire solution that we supply should be included in the system boundaries Incremental releases. A system in a UCD is represented as a box. The name of the system appears above or inside the box



ACTOR

An actor is someone or something that interacts with the system (exchanges information with the system). An actor represents a role played with respect to the system, not an individual user of the system. Example of such roles played by the actors are:

- Police man Enters data
- Supervisor allowed to modify or erase data
- Manager allowed viewing statistics

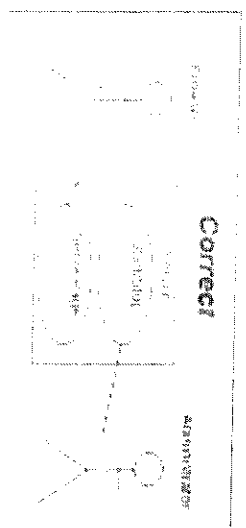
A single user may play more than one role is a fact that needs to be known by every software engineer and also the role played by an Actor is usually reflected by a name.

An actor is represented by a stickman (it is a meta-language notation used in extending a notation) <<actor>>



Use Case Icon is an ellipsis containing the name of the Use Case Placed inside the boundaries of the modelled system Connected to at least one actor with a communication association Except for specialized / extending use cases.

SAMPLE OF UCD



What Makes A Good Use Case Specification

- i. **Lack of ambiguity:** Each requirement must be interpreted in a single manner.
- ii. **Completeness:** They should cater for all current demands of the system
- iii. **Consistency:** Requirement should not conflict with each other.
- iv. **Avoid design:** requirement should raise a need and not answer it.

Benefits Of Use Cases

- It captures operational requirements from user's perspective
- Gives a clear and consistent description of what system should do.
- A basis for performing system tests
- Provides the ability to trace functional requirement into actual classes and operations in the system.
- Uncover and describe all tasks that need doing in a system of both human and system actors
- To analyse what functionality that need developing for the system
- The use of use cases must mean that the right functional requirements are made of the IT system (the requirement of the business).

c. Structural Models: structural model of software display the organization of a system in terms of the components that make up that system and their relationships. Structural models may be static models, which show the structure of the system design, or dynamic models, which show the organization of the system when it is executing. You create structural models of a system when you are discussing and designing the system architecture. Architectural design is a particularly important topic in software engineering and UML component, package, and deployment diagrams may all be used when presenting architectural models.

d. Behavioral Models: It describe the overall behaviour of the system. These are models of the dynamic behavior of the system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment. To represent system behaviour, two stimuli use one is Data processing model (data driven), and another is state machine model (event driven).

- Data. Some data arrives that has to be processed by the system. They are controlled by the data input to the system, with relatively little external event processing. **Data-driven models** show the sequence of actions involved in processing input data and generating an associated output. They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system.

- Events. Some event happens that triggers system processing. Events may have associated data but this is not always the case. **Event-driven models** shows how a system responds to external and internal events. It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another.

Principles of modeling are as follows

1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped: This means you need to choose your models well and correct as per the requirement of problem statement. Wrong model will mislead you, causing you to focus on irrelevant issues.
2. Every model may be expressed at different levels of precision: Sometimes, a quick and simple executable model of the user interface is exactly what you need. The best kinds of models are those that let you choose your degree of detail, depending on who is viewing it. This means all the user and developers both may visualize a system at different levels of details at different time.
3. The best models are connected to reality: This means that the model must have things that are practically possible. They must satisfy the real word scenarios. i.e the gap between the analysis model and the system's design model must be less.
4. No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models: here you need to understand the architecture of the system, as such you need to have use case view (exposing the requirements of the system), design view (capturing the vocabulary of the problem space and the solution space), a process view (modeling the distribution of the system's processes and threads), an implementation view (addressing the physical realization of the system) and development view (focusing on system engineering issues). Each of these views may have structural as well as behavioral aspects. Together these views represent a system.

Modeling Notations Used In Requirement Phase Of Software Development

There are infinite numbers of notations used daily. new notations are introduced to the software market, although some of these notations have similar look and feel.

These notations include:

- Entity-Relationship Diagrams
- Event Traces
- State Machines
- Data-Flow Diagrams

Entity-Relationship Diagrams (ERD)

Entity-relationship diagram is a database modelling method, used to produce a type of conceptual diagram or semantic data model of a system. The first stage of information system

design uses these notations during the requirement analysis to describe information needs that is to be stored in a database.

ERD have three (3) building blocks that are combined to specify a problem's elements and their interrelationships:

- Entities
- Relationships
- Attributes

An **entity** may be defined as a thing which is recognized as being capable of an independent existence (the basic conceptual or real element) and which can be uniquely identified. An entity may be a physical object such as a house or a car, an event such as a house sale or a car service, or a concept such as a customer transaction or order. It can be an entity and an entity-type. An entity-type is a category. An entity is an instance of a given entity-type.

Entities can be thought of as nouns. Examples: an employee, a student, a song. Each entity is associated with a set of attributes. The instances of an entity have the same characteristics. They have the same set of attributes. Examples

- All students have the same set of attributes
- All members of staff have the same set of attributes

Entities are represented as rectangles.



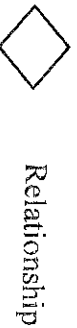
A **relationship** captures how two or more entities are related to one another. Relationships can be thought of as verbs, linking two or more nouns. Examples:

- an owns relationship between a company and a computer,
- a supervises relationship between an employee and a department,
- a performs relationship between an artist and a song,
- a proved relationship between a mathematician and a theorem.

There are four (4) kinds of relationships available.

- i. Dependency
- ii. Association
- iii. Generalization
- iv. Realization

Relationships are represented as diamonds, connected by lines to each of the entities in the relationship.



Dependency is a relationship between two things in which change in one element also affects the other.



Association is basically a set of links that connects the elements of a UML model. It also describes how many objects are taking part in that relationship.



Generalisation can be defined as a relationship which connects a specialized element with a generalised element. It basically describes the inheritance relationship in the world of objects.



Realization can be defined as a relationship in which two elements are connected. One element describes some responsibility, which is not implemented and the other one implements them. This relationship exists in case of interfaces.



Attributes are used to describe the entities. The entities and their attributes are determined according to the database project, taking into account the functions to be accomplished. Attributes are drawn as ovals and are connected with a line to exactly one entity or relationship set. Examples:

- Represent students at the department for course registration
- Represents members of staff for salaries and promotions
- One of the attributes must be an IDENTIFIER
- Its value is unique for each entity



Entities and relationships can both have attributes. Examples:

- an employee entity might have a Social Security Number (SSN) attribute;
- the proved relationship may have a date attribute.



An entity with an attribute



A relationship with an attribute

If an entity set participates in a relationship set, they are connected with a line



Two related entities

Entity-relationship diagrams don't show single entities or single instances of relations. Rather, they show entity sets and relationship sets. Example: a particular song is an entity. The collection of all songs in a database is an entity set. The eaten relationship between a child and her lunch is a single relationship. The set of all such child-lunch relationships in a database is a relationship set.

Event Traces

An event trace is a graphical description of a sequence of events that are exchanged between real-world entities; it provides developers with information useful for debugging; provides description of require behaviour or event. Event traces are popular among both developers and customers because traces have a semantics that is relatively precise, with the exception of timing issues, yet is simple and easy to understand. This simplicity comes from decomposing requirements description into scenarios and considering (modelling, reading, and understanding) each scenario separately as a distinct trace. Because of these same properties, event traces are not very efficient for documenting behaviour. Traces are best used at the start of a project, to come to consensus on key requirements and to help developers identify important entities in the problem being modelled.

State Machine

State machine is useful when the state of an object in its life cycle is important. It defines the sequence of states an object goes through in response to events. Events are external factors responsible for state change

Data Flow Diagram:

It is used to model the system data processing, also known as a functional model, as it is a graphical representation of an enterprise function within a defined scope. It shows end to end processing of data. It takes an input – process – output view of the system. It can be easily converted into software as they just represent the flow of data objects. This diagram enables a software engineer to develop a model of the information domain and Functional domain at the same time. The data processing model is the core modelling activity in structured analysis.

Overview Of Unified Modelling Language (UML)

The Unified Modelling Language (UML) is a graphical language for visualizing, specifying, constructing and documenting the artifacts of a software. The UML offers a standard way to write a system's blueprints, including conceptual things such as business processes and system functions as well as concrete things such as programming language statements, database schemas, and reusable software components.

One of the purposes of UML was to provide the development community with a stable and common design language that could be used to develop and build computer applications. UML brought forth a unified standard modelling notation that IT professionals had been wanting for years. UML combines the best practice from data modelling concepts such as entity relationship diagrams, business modelling (work flow), object modelling and component modelling. It can be used with all processes, throughout the software development life cycle, and across different implementation technologies.

UML is not a programming language but an object-oriented unified modeling language., it is rather a visual language. We use UML diagrams to portray the behavior and structure of a system. UML helps software engineers, businessmen and system architects with modelling, design and analysis. The Object Management Group (OMG) adopted Unified Modelling Language as a standard in 1997. Its been managed by OMG ever since. International Organization for Standardization (ISO) published UML as an approved standard in 2005.

The main purposes (uses) of UML can be summarized as follows:

Provide users with a ready-made, expressive visual modeling language so they can develop and communicate meaningful models in a collaborative team effort.

Provides extensibility and specialization mechanisms for core concepts.

Independent of a specific programming language and development process.

Provides a formal foundation for understanding modeling languages.

Encourage the development of the market for object-oriented tools.

Support for higher level development concepts such as collaboration, frameworks, patterns and components.

Integrate Best Practices.

UML has **13 types of diagrams** divided into three categories:

- Six diagram types represent structure application structure,
- three represent general types of behavior, and
- Four represent different aspects of interactions.

Structure diagrams: Structure diagrams emphasize what things must be in the system being modeled

1. Class diagram: describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the objects.
2. Component diagram: shows how a software component is split up to interact and shows the dependencies among these components.
3. Composite structure diagram: describes the internal structure of a class and the collaborations that this structure makes possible.
4. Deployment diagram: this tells us what hardware components exist and what software components run on them. It serves to model the hardware used in system implementations, the components deployed on the hardware, and the associations among those components.
5. Object diagram: shows a complete or partial view of the structure of a modeled system at a specific time.

6. Package diagram: depicts how a system is split up into logical groupings by showing the dependencies among these groupings.

Behavior diagrams Behavior diagrams emphasize what must happen in the system being modeled

- 1.. Activity diagram: represents the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.
- 2.. State diagram: standardized notation to describe many systems, from computer programs to business processes.
- 3.. Use case diagram: shows the functionality provided by a system and its interaction with external agents (actors).

Interaction diagrams Interaction diagrams, a subset of behavior diagrams, emphasize the flow of control and data among the things in the system being modelled:

- 1.. Communication diagram: shows the interactions between objects or parts in terms of sequenced messages. They represent a combination of information taken from Class, Sequence, and Use Case Diagrams describing both the static structure and dynamic behavior of a system.
- 2.. Interaction overview diagram: a type of activity diagram in which the nodes represent interaction diagrams.
3. Sequence diagram: shows how objects communicate with each other in a sequential order.
4. Timing diagrams: are a specific type of interaction diagram, where the focus is on timing constraint. i.e is used to depict the behavior of objects over a time frame.

In summary,

- Structural diagram - represents the framework for the system and this framework is the place where all other components exist. While
- Behavioral diagram - describes the interaction in the system. It represents the interaction among the structural diagrams. While
- Interaction diagram - represents the overall framework of the system. It contains both structural and behavioral elements of the system. or interaction diagram can be defined as the blueprint of the entire system.

Types of UML Software: MagicDraw, ArgoUML, Altova, Gliffy, LucidChart, MS Visio etc

EXPLAIN OBJECT-ORIENTED MODELLING

Object Oriented(OO) Analysis and Design

OO analysis can be defined as an investigation objects. Design means collaboration of identified objects.

Thus, it is important to understand the OO analysis and design concepts. The most important purpose of OO analysis is to identify objects of a system to be designed. An efficient analysis is

only possible when we are able to start thinking in a way where objects can be identified. After identifying the objects, their relationships are identified and finally the design is produced.

The purpose of OO analysis and design are

- Identifying the objects of a system.
- Identifying their relationships.
- Making a design, which can be converted to executables using OO languages.
- There are three basic steps where the OO concepts are applied and implemented. The steps can be defined as: OO Analysis → OO Design → OO implementation using OO languages

The three (3) points mentioned above are described as:

During **OO analysis**, the most important purpose is to identify objects and describe them in a proper way. If these objects are identified efficiently, then the next job of design is easy. The objects should be identified with responsibilities. Responsibilities are the functions performed by the object. Each and every object has some type of responsibilities to be performed. When these responsibilities are collaborated, the purpose of the system is fulfilled.

OO design: this phase, emphasis is placed on the requirements and their fulfilment. In this stage, the objects are collaborated according to their intended association. After the association is complete, the design is also complete.

OO implementation. In this phase, the design is implemented using OO languages such as Java, C++, etc.

EXPLANATION OF OBJECT ORIENTED SYSTEM CONCEPTS

- **Objects** – Objects represent an entity and the basic building block.
- **Class** – Class is the blue print of an object. i.e. structure and functions of an object.
- **Abstraction** – Abstraction represents the behavior of a real-world entity. (Mechanism by which implementation details are hidden from user.)
- **Encapsulation** – Encapsulation is the mechanism of binding data together and hiding them from the outside world.
- **Inheritance** – Inheritance is the mechanism of making new classes from existing ones.
- **Polymorphism** – It defines the mechanism by which functions or entities are able to exist in different forms.
- **Component** – Component describes the physical part of a system.
- **Interface** – Interface defines a set of operations, which specify the responsibility of a class.

STATE MACHINES

A **State Machine** therefore is that entity that models the behaviour of a reactive object. A State Machine tends to be used for modelling the life cycle or history of a reactive object. There are

three (3) key members of a valid and working state machine and they are states, events and transitions.

- **State** – is the mode or condition of being, this is a condition or situation during the life cycle of an object during which it satisfies some conditions, performs activity or waits for an event.
- **Events** – this is a specification of a noteworthy occurrence that has location in time and space.
- **Transitions** – this is the movement from one state to another in response to an event.

Types Of State Machines

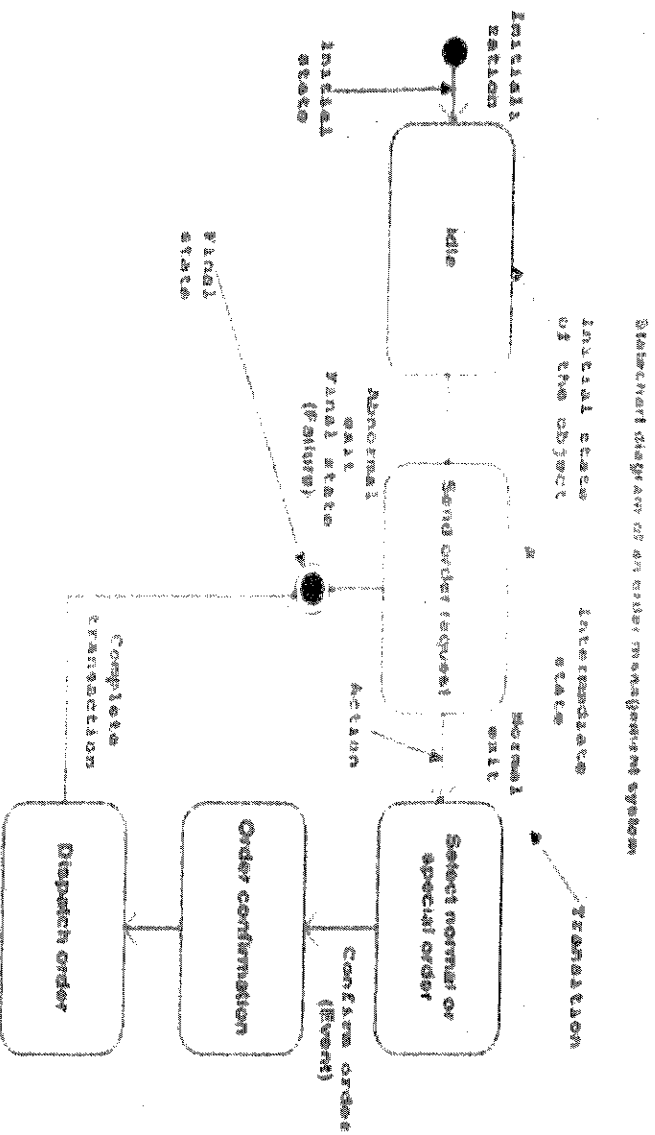
- **Behavioral State Machines** – uses states, events and transitions to define the behavior of the context classifier. It can only be used when the context classifier has a behavior of some sort to be modeled. States in behavioral state machines can specify one or more actions that execute when the state is entered, resided in or exited.
- **Protocol State Machines** – only define how the behavior appears to an external entity. They can be used to define the protocol of all classifiers. They also use states, events and transitions to define the protocol of the context classifier. The protocols are as follows:
 - Conditions under which operations may be called on the classifier and instances
 - Result of operation calls
 - Ordering of operation calls

Generally, State Machines are most commonly used to model the dynamic behaviour of classes. Each class can have a single behavioral state machine that models all the possible states, events and transitions for all instances of that class. Each class can also have one or more protocol state machines, although these are more often used with behaviourless classifiers such as interfaces and ports. The behavioural and protocol state machine for a class should specify the behaviour and protocol required by all use cases in which instances of the class participate. If a use case requires a protocol or behaviour that is not captured in a State Machine, this indicates that the State Machine are in some way incomplete.

Below information describe a scenario of a Statechart diagram which consist of four activities with some condition.

- Customer Send Order,
- Receipt of Order,
- Confirm the Order, and
- Dispatch the Order.

The Conditions: After receiving the Order request, condition checks are performed to check if it is a Special or Normal Order. After the type of Order is identified, dispatch activity is performed which marks the termination of the whole process.



In summary, a Statechart diagram describes the flow of control from one state to another state. States are defined as a condition in which an object exists and it changes when some event is triggered.

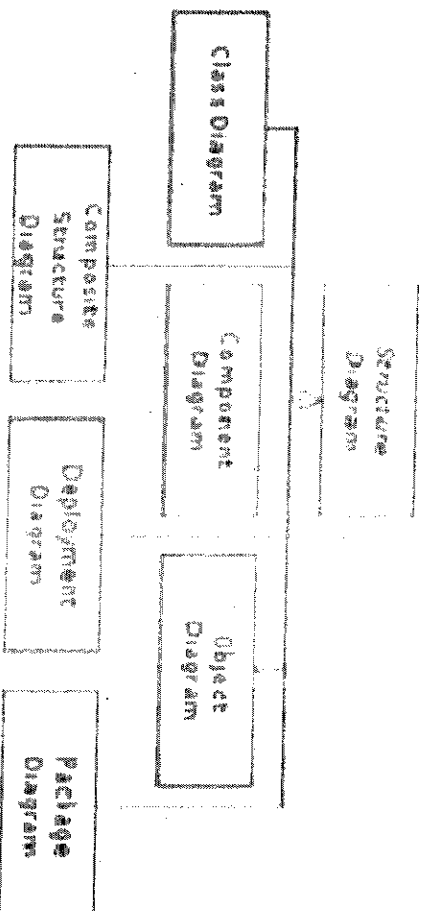
Four (4) main purposes of using statechart diagram are:

- i. To model the dynamic aspect of a system.
- ii. To model the life time of a reactive system.
- iii. To describe different states of an object during its life time.
- iv. Define a state machine to model the states of an object.

Conceptual Model

A conceptual model can be defined as a model which is made of concepts and their relationships. i.e it is a way of describing and representing some type of real-world system or concept in order to make it easier to understand. Conceptual models are often visual representations of abstract concepts.

A conceptual diagram is a visual representation of the ways in which abstract concepts are related. They are an effective tool to communicate complex messages in a simple and informative manner. It is used as an aid in visualizing processes or systems at a high level through a series of unique lines and shapes.



Class diagram

The Class diagram describes the structure of a system by showing the system's classes, their attributes, and the relationships among the classes

The basics of Class Diagram

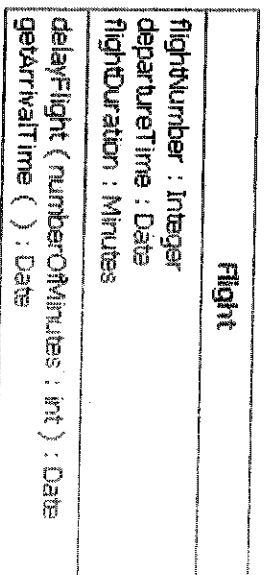
It is more important than ever in UML to understand the basics of the class diagram. This is because the class diagram provides the basic building blocks for all other structure diagrams, such as the component or object diagrams.

As mentioned earlier, the purpose of the class diagram is to show the types being modelled within the system.

In most UML models these types:

- a class
- an interface
- a data type
- a component.

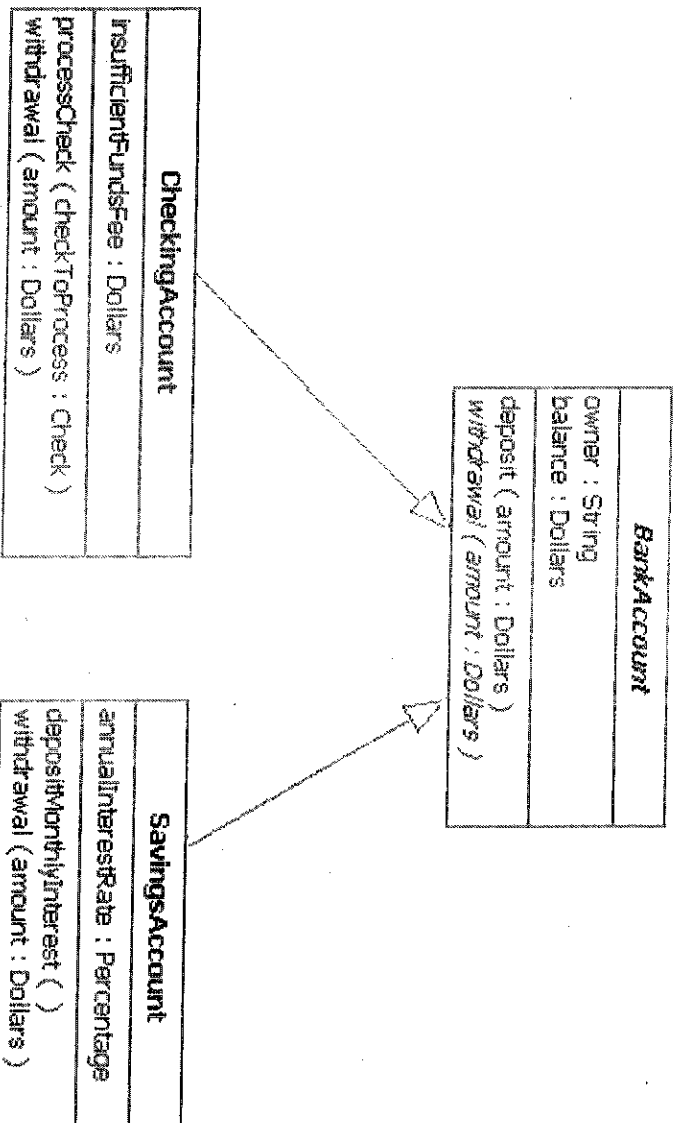
Figure 1: Class diagram for the class Flight



Class attribute list

The attribute section of a class (the middle compartment) lists each of the class's attributes on a separate line. The attribute section is optional, but when used it contains each attribute of the class displayed in a list format. The line uses the following format:

Figure below: Inheritance is indicated by a solid line with a closed, unfilled arrowhead pointing at the super class using tree notation.



Abstract classes and operations

The observant reader will notice that the diagrams in Figures above use italicized text for the BankAccount class name and withdrawal operation. This indicates that the BankAccount class is an abstract class and the withdrawal method is an abstract operation. In other words, the BankAccount class provides the abstract operation signature of withdrawal and the two child classes of CheckingAccount and SavingsAccount each implement their own version of that operation. However, super classes (parent classes) do not have to be abstract classes. It is normal for a standard class to be a super class.

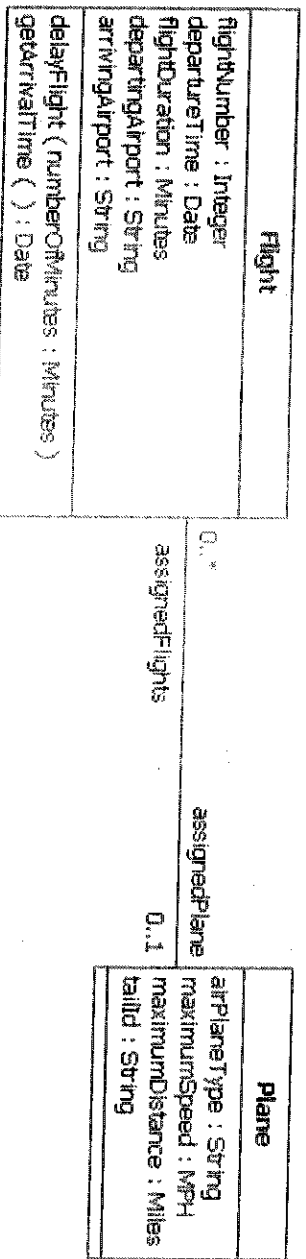
Associations

When you model a system, certain objects will be related to each other, and these relationships themselves need to be modelled for clarity. There are five types of associations.

- i. Bi-directional,
- ii. uni-directional associations,
- iii. Association class,
- iv. Aggregation, and
- v. Reflexive associations

Bi-directional (standard) association

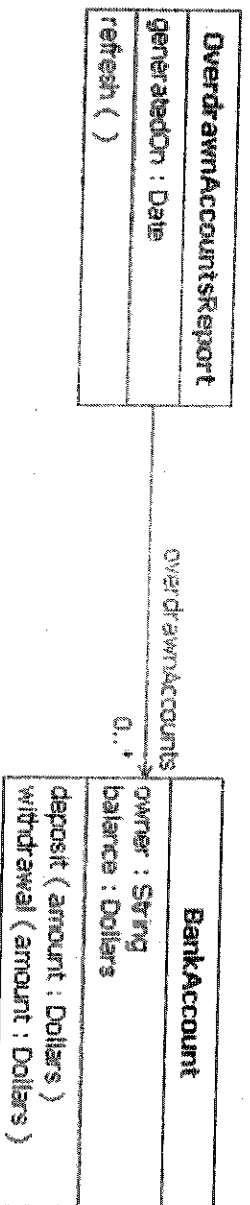
An association is a linkage between two classes. Associations are always assumed to be bi-directional; this means that both classes are aware of each other and their relationship, unless you qualify the association as some other type. Figure below shows a standard kind of association between the Flight class and the Plane class.



A bi-directional association is indicated by a solid line between the two classes. At either end of the line, you place a role name and a multiplicity value. Figure above shows that the Flight is associated with a specific Plane, and the Flight class knows about this association. The Plane takes on the role of "assignedPlane" in this association because the role name next to the Plane class says so. The multiplicity value next to the Plane class of 0..1 means that when an instance of a Flight exists, it can either have one instance of a Plane associated with it or no Planes associated with it (i.e., maybe a plane has not yet been assigned). Figure above also shows that a Plane knows about its association with the Flight class. In this association, the Flight takes on the role of "assignedFlights"; the diagram in Figure above tells us that the Plane instance can be associated either with no flights (e.g., it's a brand new plane) or with up to an infinite number of flights (e.g., the plane has been in commission for the last five years).

Uni-directional association

In a uni-directional association, two classes are related, but only one class knows that the relationship exists. Figure below shows an example of an overdrawn accounts report with a uni-directional association.



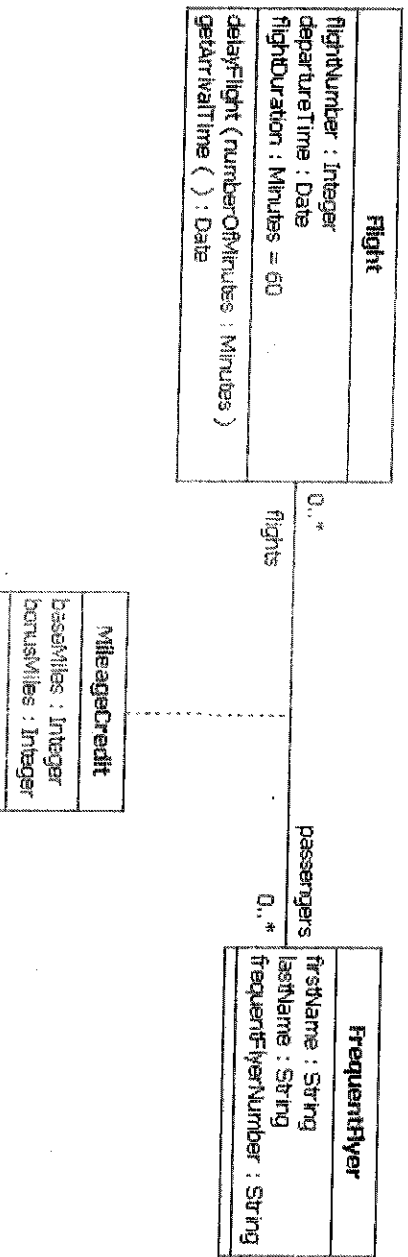
The OverdrawnAccountsReport class knows about the BankAccount class, but the BankAccount class does not know about the association.

A uni-directional association is drawn as a solid line with an open arrowhead (not the closed arrowhead, or triangle, used to indicate inheritance) pointing to the known class. Like standard associations, the uni-directional association includes a role name and a multiplicity value, but unlike the standard bidirectional association, the uni-directional association only contains the role name and multiplicity value for the known class.

Association class

In modeling an association, there are times when you need to include another class because it includes valuable information about the relationship. For this you would use an association class that you tie to the primary association. An association class is represented like a normal class. The difference is that the association line between the primary classes intersects a dotted line connected to the association class. Figure below shows an association class for airline industry

Example of Adding the association class MileageCredit

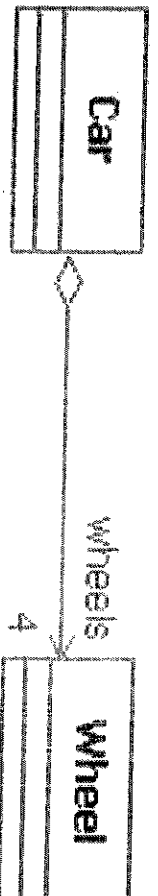


Aggregation is a special type of association used to model a "whole to its parts" relationship. In basic aggregation relationships, the lifecycle of a part class is independent from the whole class's lifecycle. For example, we can think of Car as a whole entity and Car Wheel as part of the overall Car. The wheel can be created weeks ahead of time, and it can sit in a warehouse before being placed on a car during assembly. In this example, the Wheel class's instance clearly lives independently of the Car class's instance. Consider, for example, the relationship of a company to its departments. Both Company and Departments are modelled as classes, and a department cannot exist before a company exists. Here the Department class's instance is dependent upon the existence of the Company class's instance. Let's explore basic aggregation and composition aggregation further.

Basic aggregation

An association with an aggregation relationship indicates that one class is a part of another class. In an aggregation relationship, the child class instance can outlive its parent class. To represent an aggregation relationship, you draw a solid line from the parent class to the part class, and draw an unfilled diamond shape on the parent class's association end. Figure below shows an example of an aggregation relationship between a Car and a Wheel.

: Example of an aggregation association



Composition aggregation

The composition aggregation relationship is just another form of the aggregation relationship, but the child class's instance lifecycle is dependent on the parent class's instance lifecycle. In Figure below, which shows a composition relationship between a Company class and a Department class, notice that the composition relationship is drawn like the aggregation relationship, but this time the diamond shape is filled.

Example of a composition relationship

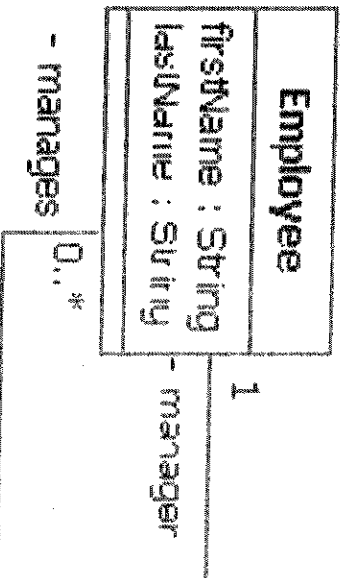


In the relationship modelled in Figure above, a Company class instance will always have at least one Department class instance. Because the relationship is a composition relationship, when the Company instance is removed/destroyed, the Department instance is automatically removed/destroyed as well. Another important feature of composition aggregation is that the part class can only be related to one instance of the parent class.

Reflexive associations

A class can also be associated with itself, below shows how an Employee class could be related to itself through the manager/manages role. When a class is associated to itself, this does not mean that a class's instance is related to itself, but that an instance of the class is related to another instance of the class.

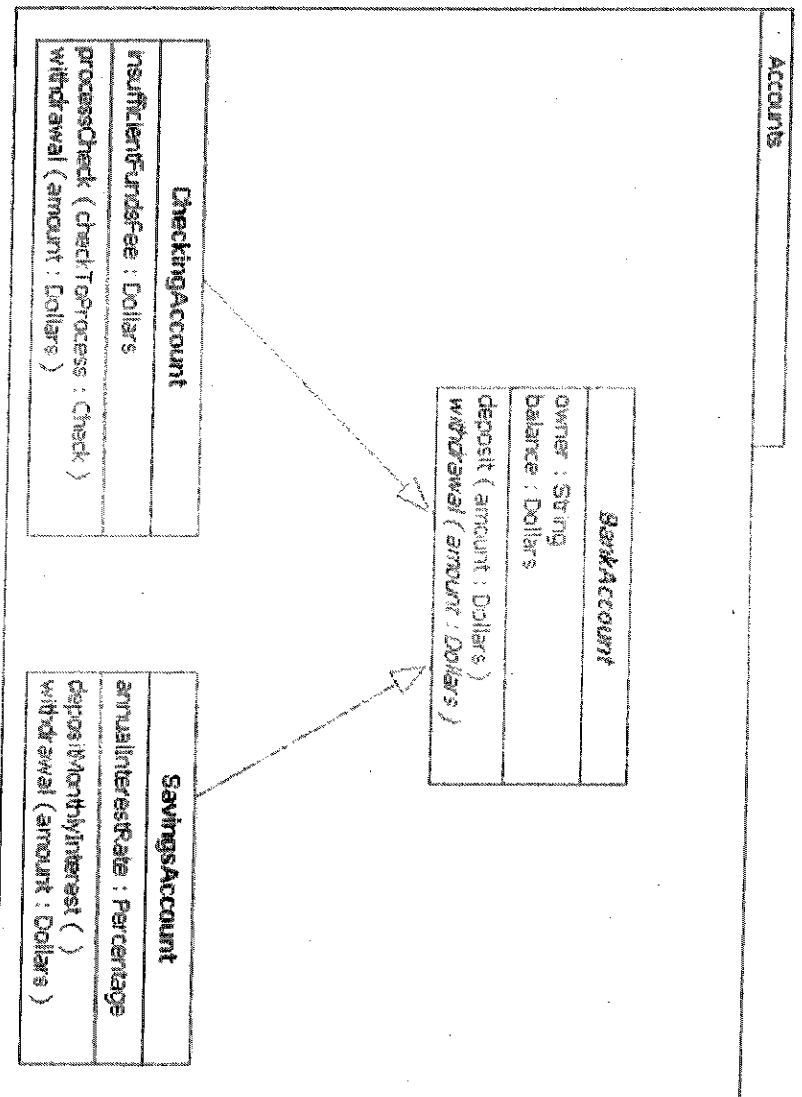
Example of a reflexive association relationship



Packages

Packages enable modellers to organize the model's classifiers into namespaces, which is sort of like folders in a filing system. Dividing a system into multiple packages makes the system easier to understand, especially if each package represents a specific part of the system.

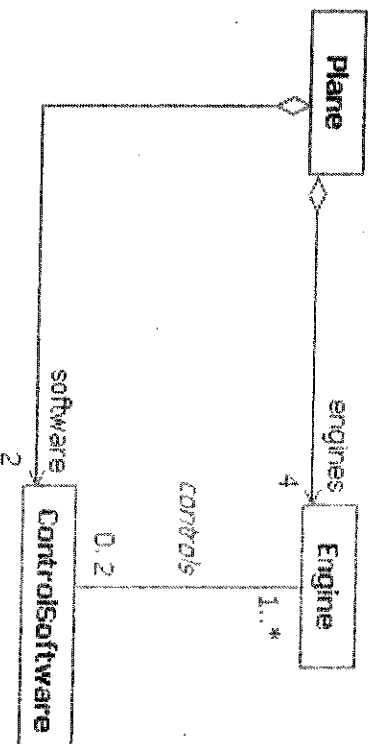
An example package element that shows its members inside the package's rectangle boundaries



Visibility : In object-oriented design, visibility for attributes and operations exist. UML identifies four types of visibility: public, protected, private, and package.

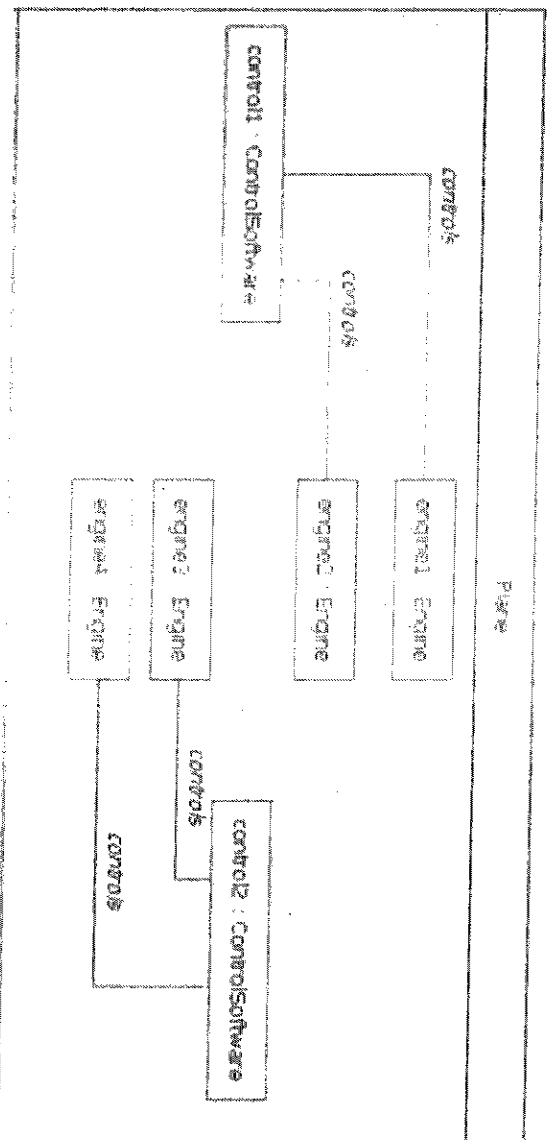
Internal structure : One of the more useful features of UML structure diagrams is the new internal structure notation. It allows you to show how a class or another classifier is internally composed.

A class diagram that only shows relationships between the objects



Drawing a class's internal structure will improve this situation. You start by drawing a box with two compartments. The top compartment contains the class name, and the lower compartment contains the class's internal structure, showing the parent class's part classes in their respective roles, as well as how each particular class relates to others in that role.

An example internal structure of a Plane class.



In Figure above the Plane has two ControlSoftware objects and each one controls two engines. The ControlSoftware on the left side of the diagram (control1) controls engines 1 and 2. The ControlSoftware on the right side of the diagram (control2) controls engines 3 and 4.

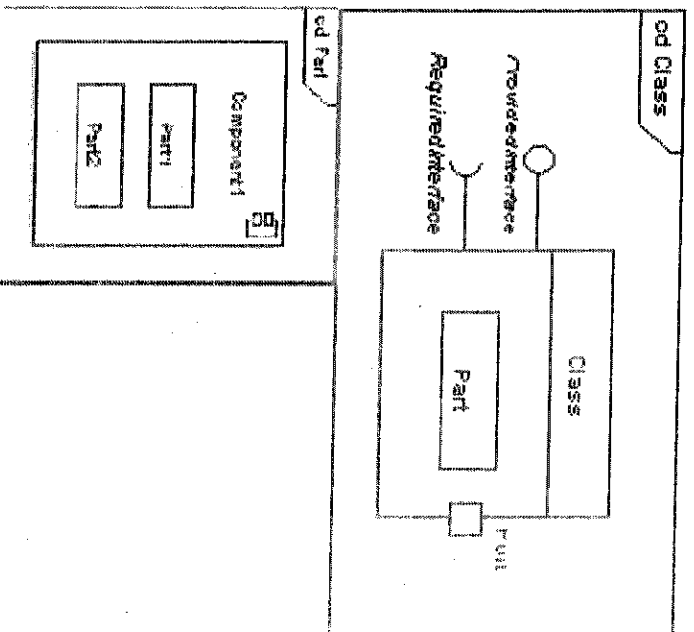
COMPOSITE DIAGRAM

A composite structure diagram is a diagram that shows the internal structure of a classifier, including its interaction points to other parts of the system. It shows the configuration and relationship of parts that together, perform the behavior of the containing classifier.

This section describes the way classes can be displayed as composite elements exposing interfaces and containing ports and parts.

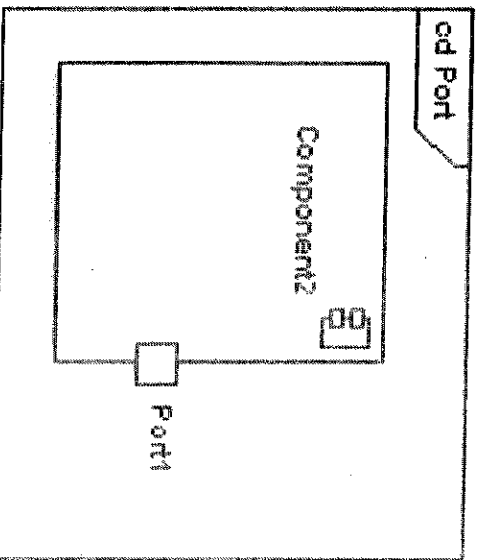
Part : A part is an element that represents a set of one or more instances which are owned by a containing classifier instance. So for example, if a diagram instance owned a set of graphical elements, then the graphical elements could be represented as parts; if it were useful to do so, to model some kind of relationship between them. Note that a part can be removed from its parent before the parent is deleted, so that the part isn't deleted at the same time.

A part is shown as an unadorned rectangle contained within the body of a class or component element.



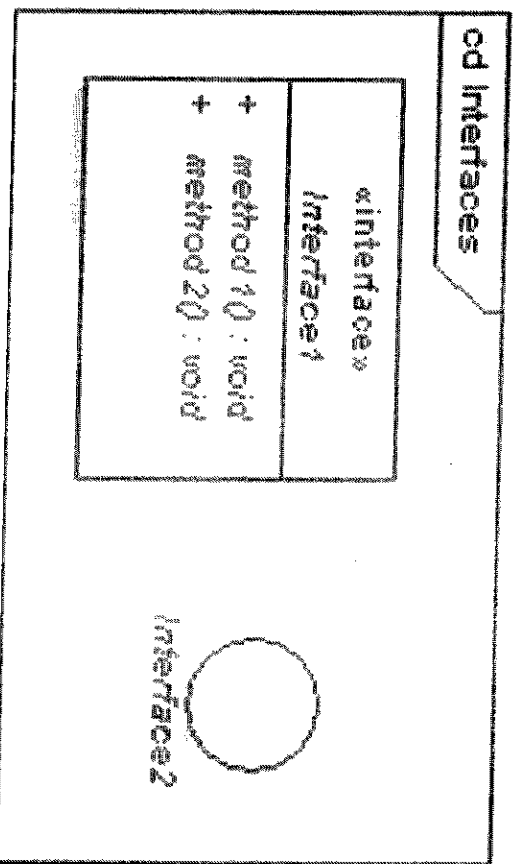
Port : A port is a typed element that represents an externally visible part of a containing classifier instance. Ports define the interaction between a classifier and its environment. A port can appear on the boundary of a contained part, a class or a composite structure. A port may specify the services a classifier provides as well as the services that it requires of its environment.

A port is shown as a named rectangle on the boundary edge of its owning classifier



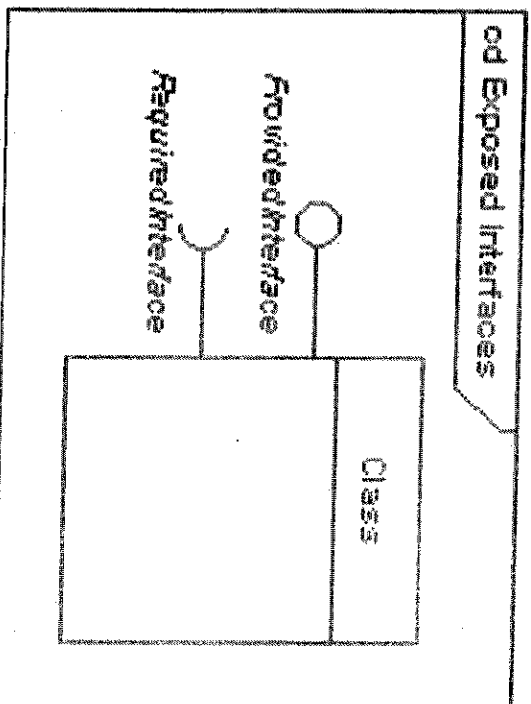
Interface : An interface is similar to a class but with a number of restrictions. All interface operations are public and abstract, and do not provide any default implementation. All interface attributes must be constants. However, while a class may only inherit from a single superclass, it may implement multiple interfaces.

An interface, when standing alone in a diagram, is either shown as a class element rectangle with the «interface» keyword and with its name italicized to denote it is abstract, or it is shown as a circle. Page 12 of 34

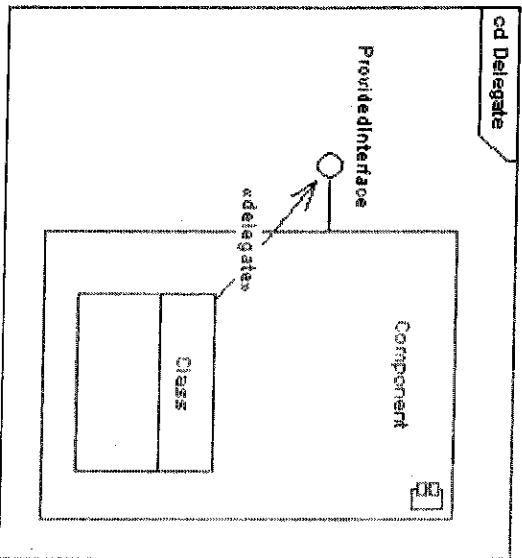


Note that the circle notation does not show the interface operations. When interfaces are shown as being owned by classes, they are referred to as exposed interfaces. An exposed interface can be defined as either provided or required. A required interface is a statement that the classifier is able to communicate with some other classifier which provides operations defined by the named interface element and is defined by drawing a dependency link between the class and the interface.

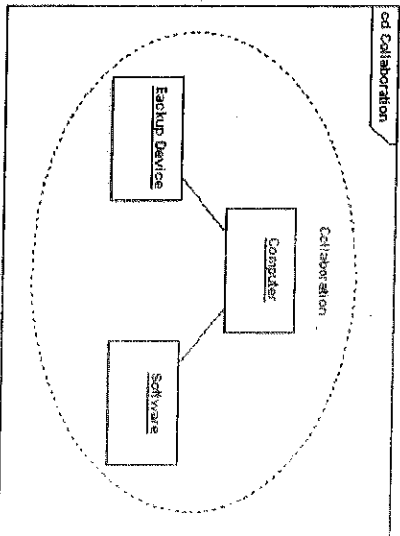
A provided interface is shown as a "ball on a stick" attached to the edge of a classifier element. A required interface is shown as a "cup on a stick" attached to the edge of a classifier element



Delegate : A delegate connector is used for defining the internal workings of a component's external ports and interfaces. A delegate connector is shown as an arrow with a «delegate» keyword. It connects an external contract of a component as shown by its ports to the internal realization of the behavior of the component's part.



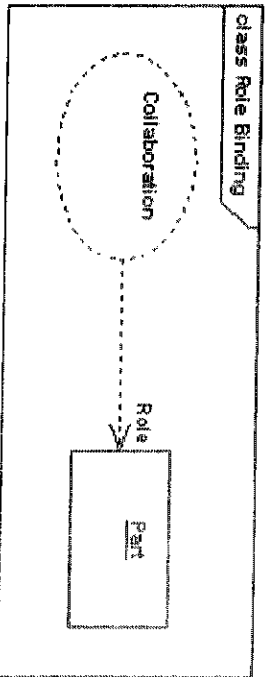
Collaboration : A collaboration defines a set of co-operating roles used collectively to illustrate a specific functionality. A collaboration should only show the roles and attributes required to accomplish its defined task or function. Isolating the primary roles is an exercise in simplifying the structure and clarifying the behavior, and also provides for re-use.



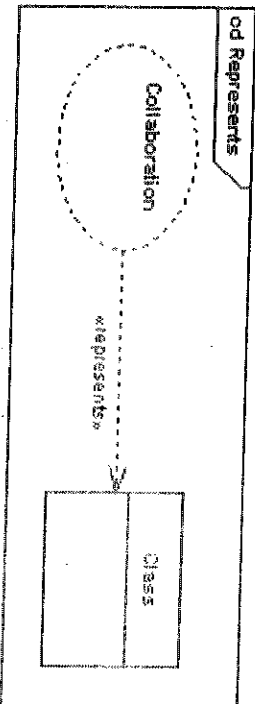
A collaboration often implements a pattern. A collaboration element is shown as an ellipse

Role Binding

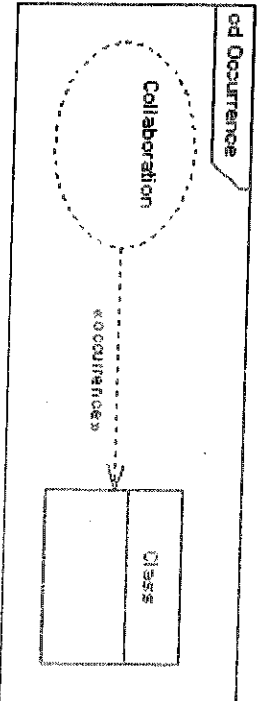
A role binding connector is drawn from collaboration to the classifier that fulfils the role. It is shown as a dashed line with the name of the role at the classifier end.



Represents : A represents connector may be drawn from a collaboration to a classifier to show that a collaboration is used in the classifier. It is shown as a dashed line with arrowhead



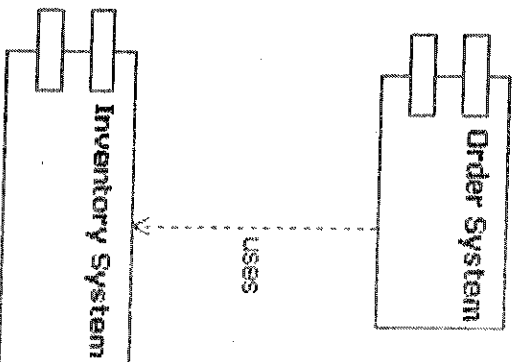
Occurrence: An occurrence connector may be drawn from collaboration to a classifier to show that collaboration represents the classifier. It is shown as a dashed line with arrowhead and the keyword «occurrence».



THE COMPONENT DIAGRAM

The component diagram's main purpose is to show the structural relationships between the components of a system.

Developers find the component diagram useful because it provides them with a high-level, architectural view of the system that they will be building, which helps developers begin formalizing a roadmap for the implementation, and make decisions about task assignments and/or needed skill enhancements. System administrators find component diagrams useful because they get an early view of the logical software components that will be running on their systems.



OBJECT AND PACKAGE DIAGRAM

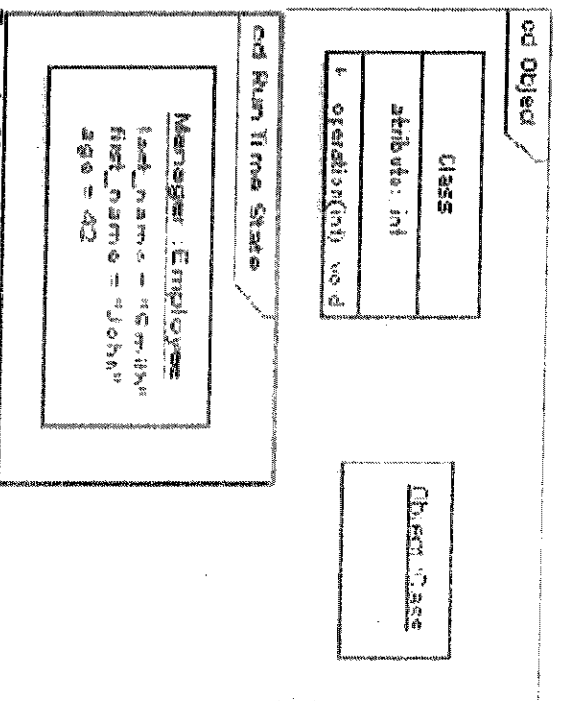
Object diagram: shows a complete or partial view of the structure of a modelled system at a specific time. An object diagram may be considered a special case of a class diagram. Object diagrams use a subset of the elements of a class diagram in order to emphasize the relationship between instances of classes at some point in time. They are useful in understanding class diagrams. They don't show anything architecturally different to class diagrams, but reflect multiplicity and roles.

Class and Object Elements

The class element consists of three parts, being divided into name, attribute and operation compartments. The display of names is also different: object names are underlined and may show the name of the classifier from which the object is instantiated.

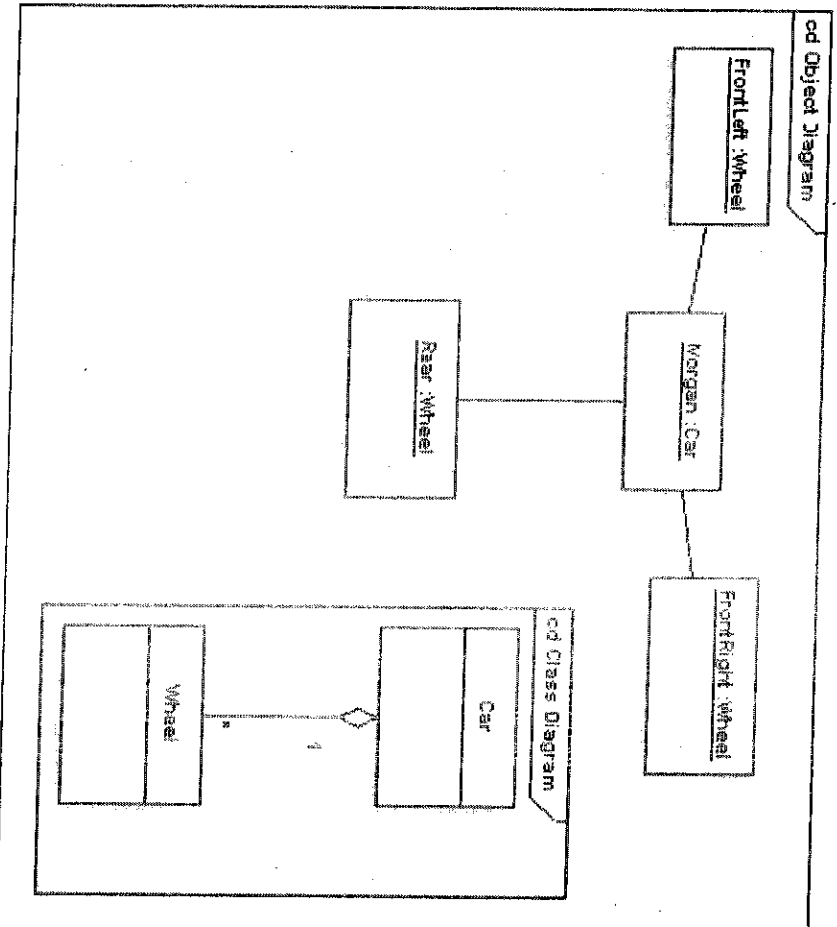
Run Time State

A classifier element can have any number of attributes and operations. These are not shown in an object instance. It is possible, however, to define an object's run time state, showing the set values of attributes in the particular instance.



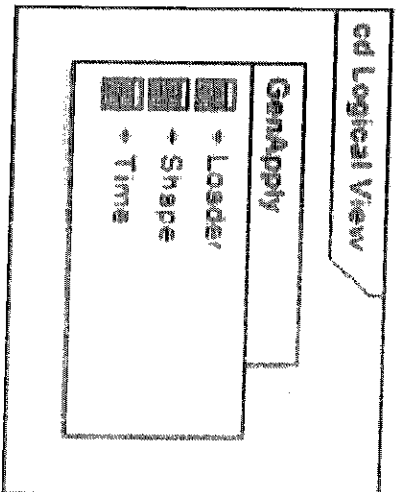
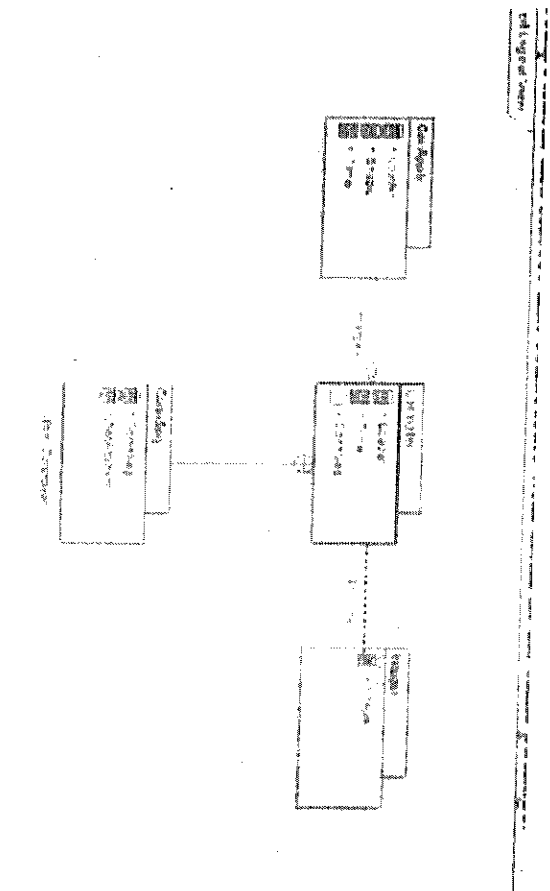
Example Class and Object Diagrams

The following diagram shows an object diagram with its defining class diagram inset, and it illustrates the way in which an object diagram may be used to test the multiplicities of assignments in class diagrams. The car class has a 1-to-many multiplicity to the wheel class, but if a 1-to-4 multiplicity had been chosen instead, that wouldn't have allowed for the three-wheeled car shown in the object diagram.



Package diagram depicts how a system is split up into logical groupings by showing the dependencies among these groupings. Package diagrams are used to reflect the organization of packages and their elements. When Page used to represent class elements, package diagrams provide a visualization of the namespaces. The most common use for package diagrams is to organize use case diagrams and class diagrams.

Elements contained in a package share the same namespace. Therefore, the elements contained in a specific namespace must have unique names. Packages can be built to represent either physical or logical relationships. When choosing to include classes in specific packages, it is useful to assign the classes with the same inheritance hierarchy to the same package. There is also a strong argument for including classes that are related via composition, and classes that collaborate with them, in the same package.



within a package must be identifiable, and so has a unique name or type. The package must show the package name and can optionally show the elements within the package in extra compartments.

Package Merge

A «merge» connector between two packages defines an implicit generalization between elements in the source package, and elements with the same name in the target package. The source element definitions are expanded to include the element definitions contained in the target. The target element definitions are unaffected, as the definitions of source package elements that don't match names with any element in the target package.

Package Import

The «import» connector indicates that the elements within the target package, which in this example is a single class, use unqualified names when being referred to from the source package. The source package's namespace gains access to the target classes; the target's namespace is not affected.

Nesting Connectors

The nesting connector between the target package and source packages shows that the source package is fully contained in the target package.

DEPLOYMENT

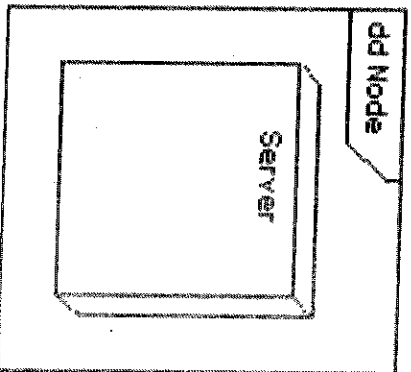
Deployment diagram serves to model the hardware used in system implementations, the components deployed on the hardware, and the associations among those components.

A deployment diagram models the run-time architecture of a system. It shows the configuration of the hardware elements (nodes) and shows how software elements and artifacts are mapped onto those nodes.

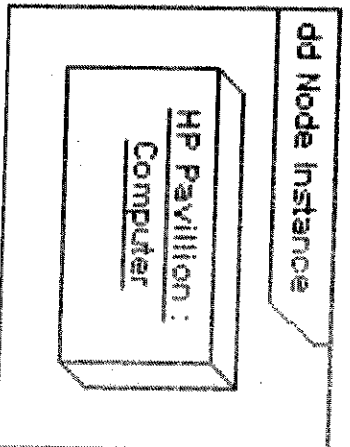
Node

A Node is either a hardware or software element. It is shown as a three-dimensional box shape, as shown below.

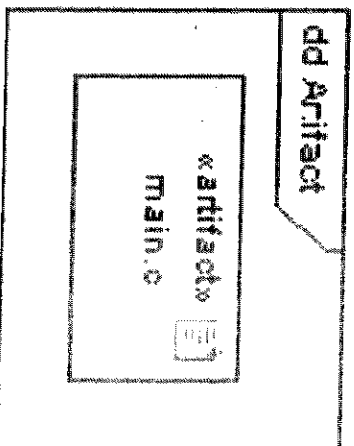
Node Instance : A node instance can be shown on a diagram. An instance can be distinguished from a node by the fact that its name is underlined and has a colon before its base node type. An instance may or may not have a name before the colon. The following diagram shows a named instance of a computer.



Node Stereotypes : A number of standard stereotypes are provided for nodes, namely «cdrom», «cdrom», «computer», «disk array», «pc», «pc client», «pc server», «secure», «server», «storage», «unix server», «user pc». These will display an appropriate icon in the top right corner of the node symbol

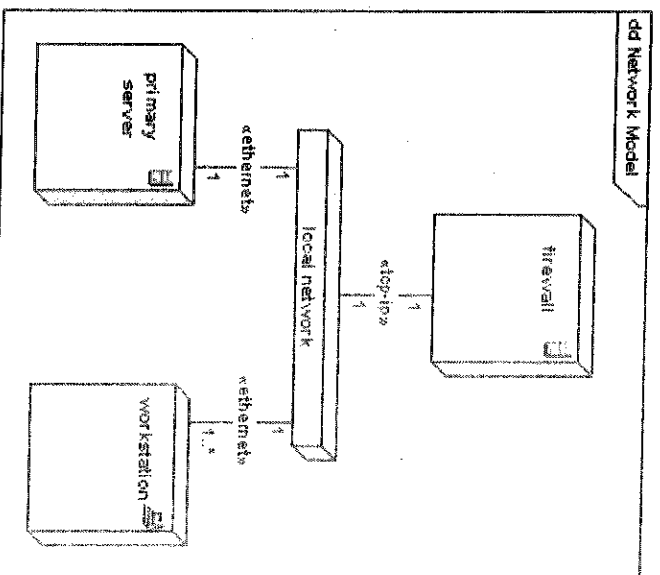


Artifact : is a product of the software development process. That may include process models (e.g. use case models, design models etc), source files, executable, design documents, test reports, prototypes, user manuals, etc. An artifact is denoted by a rectangle showing the artifact name, the «artifact» keyword and a document icon, as shown below .

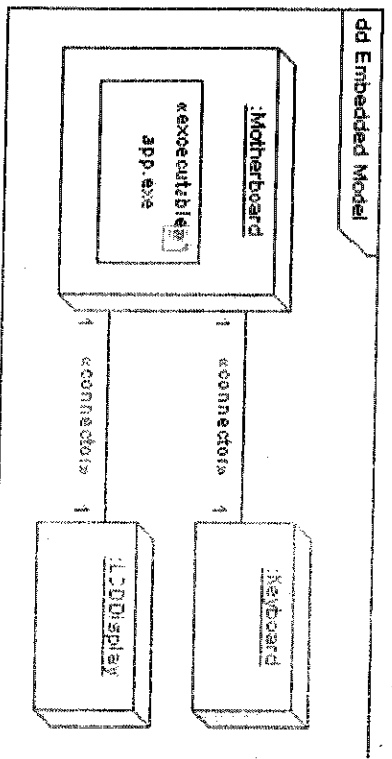


Association

In the context of a deployment diagram, an association represents a communication path between nodes. The following diagram shows a deployment diagram for a network, depicting network protocols as stereotypes, and multiplicities at the association ends.



Node as Container: A node can contain other elements, such as components or artifacts. The following diagram shows a deployment diagram for part of an embedded system, depicting an executable artifact as being contained by the motherboard node.



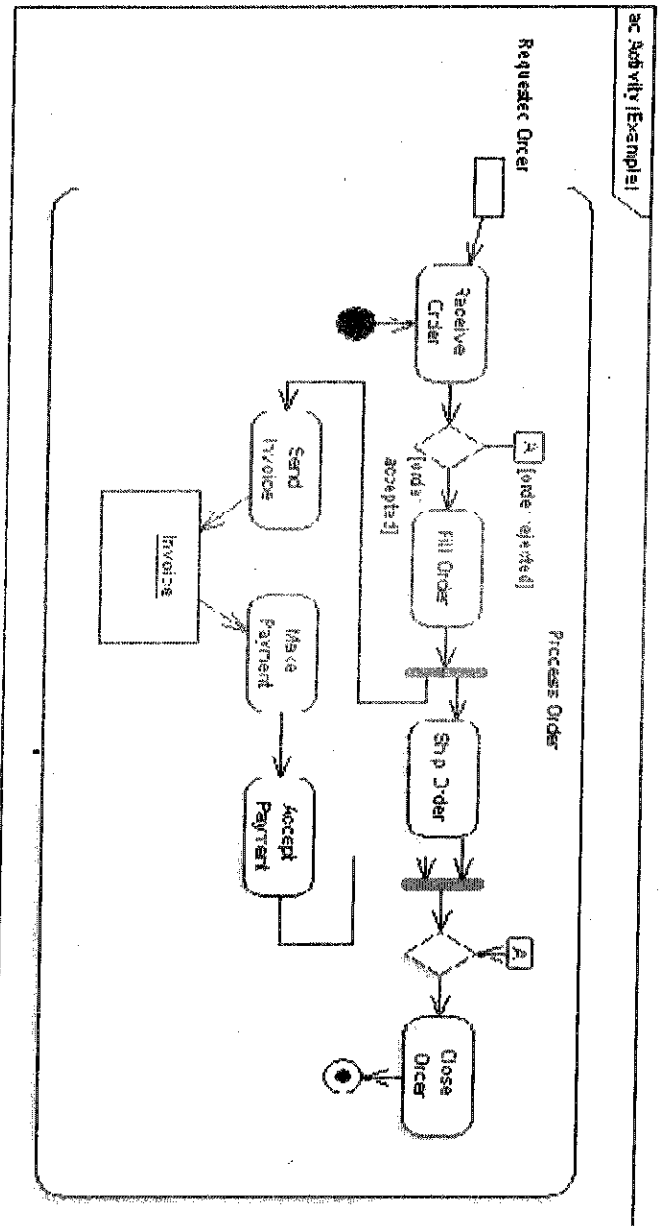
THE BEHAVIOUR DIAGRAM

The Behavior diagrams emphasize what must happen in the system being modelled.

Activity Diagrams

In UML, an activity diagram is used to display the sequence of activities. Activity diagrams show the workflow from a start point to the finish point detailing the many decision paths that exist in the progression of events contained in the activity. They may be used to detail situations where parallel processing may occur in the execution of some activities. Activity diagrams are useful for business modelling where they are used for detailing the processes involved in business activities.

An Example of an activity diagram is shown below



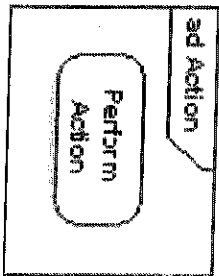
The following sections describe the elements that constitute an activity diagram.

Activities

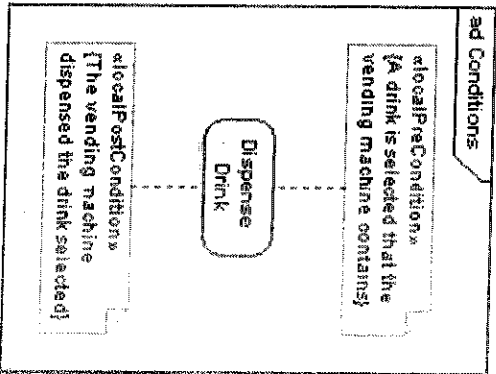
An activity is the specification of a parameterized sequence of behaviour. An activity is shown as a round-cornered rectangle enclosing all the actions, control flows and other elements that make up the activity.

Actions:

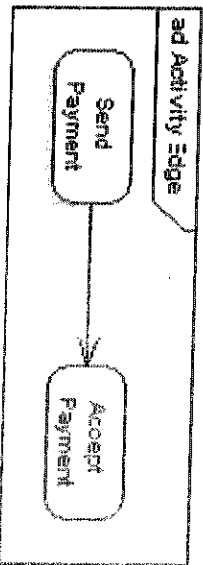
An action represents a single step within an activity. Actions are denoted by round-cornered rectangles.



Action Constraints : Constraints can be attached to an action. The following diagram shows an action with local pre-and post-conditions.

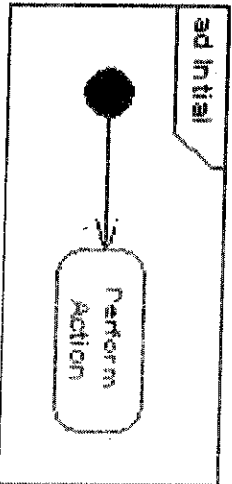


Control Flow : A control flow shows the flow of control from one action to the next. Its notation is a line with an arrowhead.



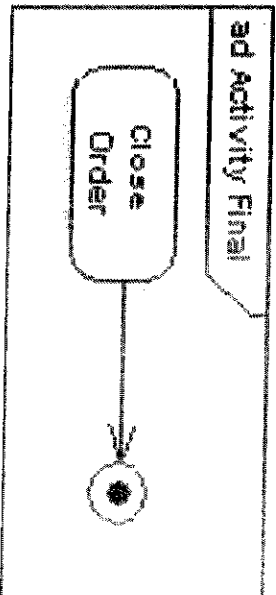
Initial Node

An initial or start node is depicted by a large black spot, as shown below.

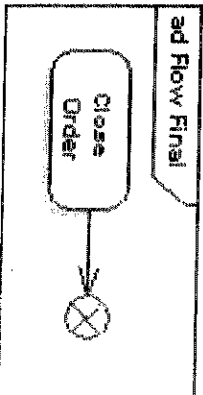


Final Node

There are two types of final node: activity and flow final nodes. The activity final node is depicted as a circle with a dot inside. The flow final node is depicted as a circle with a cross inside.

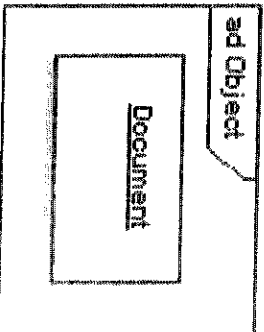


The difference between the two node types is that the flow final node denotes the end of a single control flow; the activity final node denotes the end of all control flows within the activity.

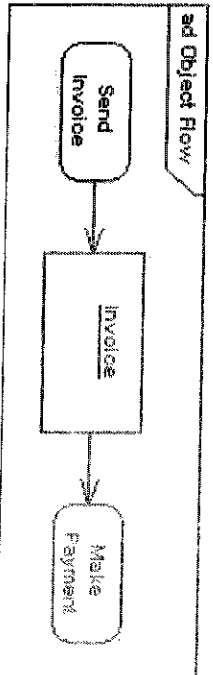


Objects and Object Flows

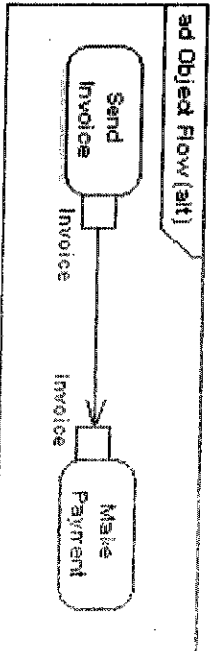
An object flow is a path along which objects or data can pass. An object is shown as a rectangle.



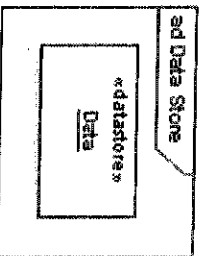
An object flow is shown as a connector with an arrowhead denoting the direction the object is being passed.



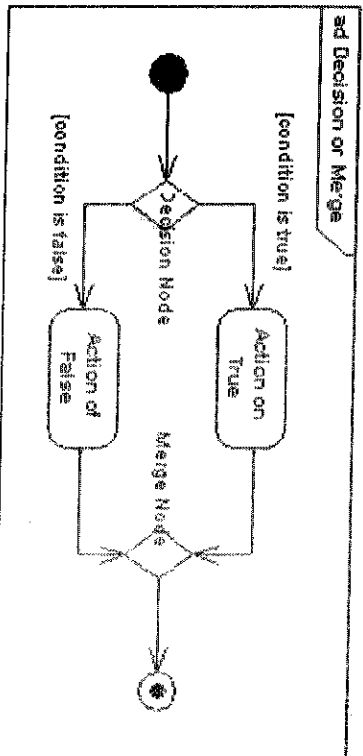
An object flow must have an object on at least one of its ends. A shorthand notation for the above diagram would be to use input and output pins.



A data store is shown as an object with the «datastore» keyword.

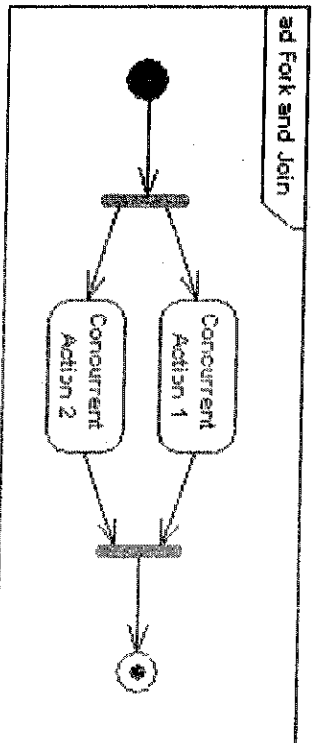


Decision and Merge Nodes : Decision nodes and merge nodes have the same notation: a diamond shape. They can both be named. The control flows coming away from a decision node will have guard conditions which will allow control to flow if the guard condition is met. The following diagram shows use of a decision node and a merge node.



Fork and Join Nodes

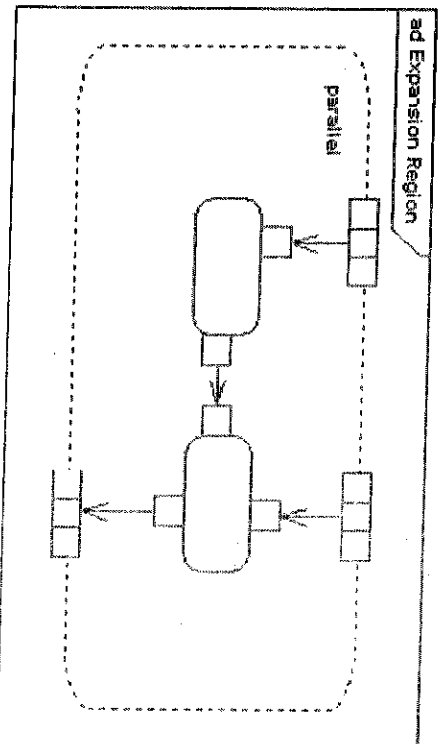
Forks and joins have the same notation: either a horizontal or vertical bar (the orientation is dependent on whether the control flow is running left to right or top to bottom). They indicate the start and end of concurrent threads of control. The following diagram shows an example of their use.



A join is different from a merge in that the join synchronizes two inflows and produces a single outflow. The outflow from a join cannot execute until all inflows have been received. A merge passes any control flows straight through it. If two or more inflows are received by a merge symbol, the action pointed to by its outflow is executed two or more times.

Expansion Region

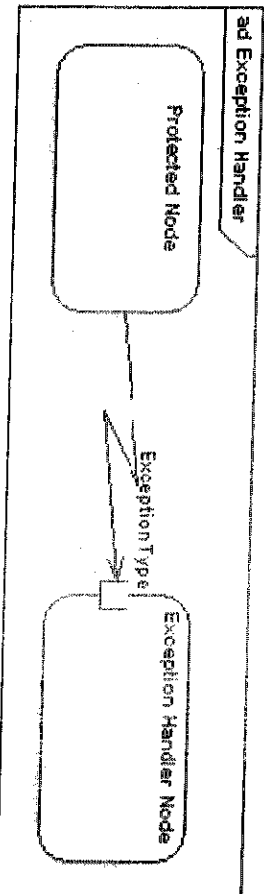
An expansion region is a structured activity region that executes multiple times. Input and output expansion nodes are drawn as a group of three boxes representing a multiple selection of items. The keyword "iterative", "parallel" or "stream" is shown in the top left corner of the region.



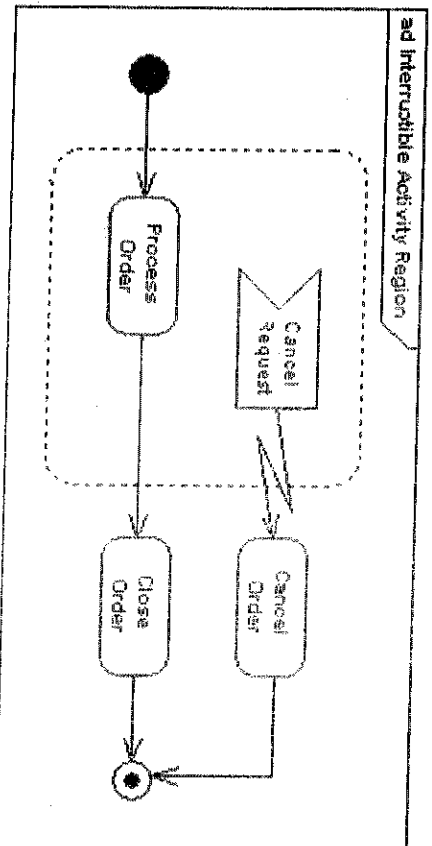
Exception Handlers

Exception Handlers can be modelled on activity diagrams as in the example below.

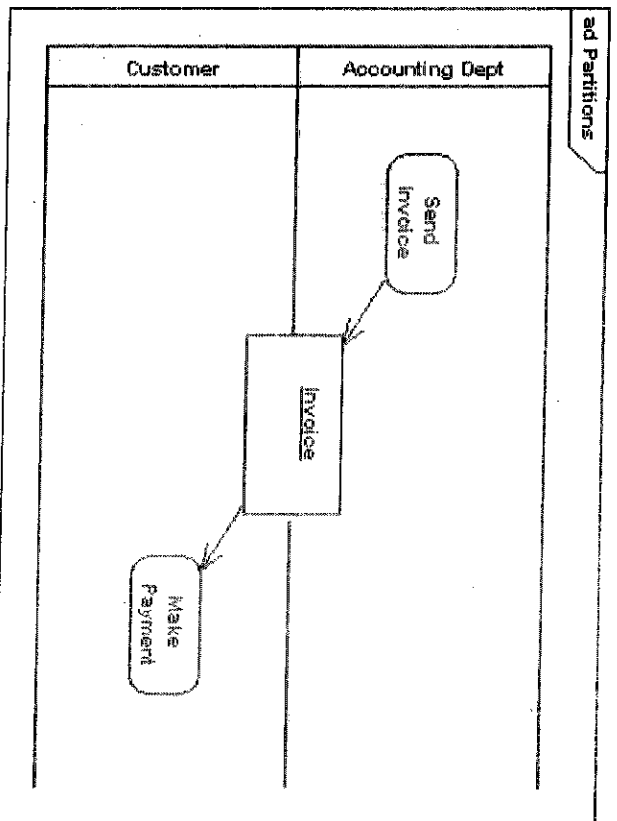
Interruptible Activity Region



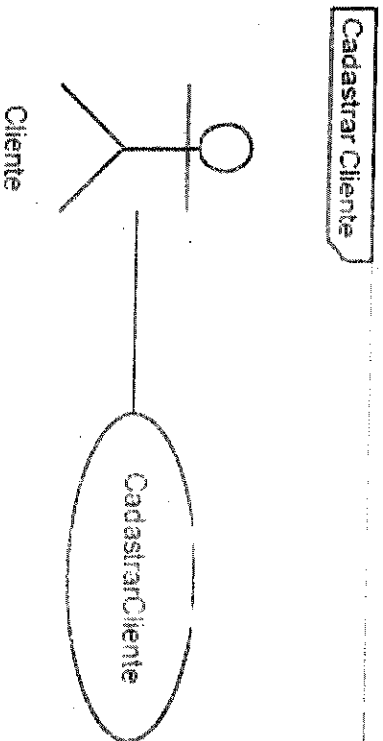
An interruptible activity region surrounds a group of actions that can be interrupted. In the very simple example below, the "Process Order" action will execute until completion, when it will pass control to the "Close Order" action, unless a "Cancel Request" interrupt is received, which will pass control to the "Cancel Order" action.



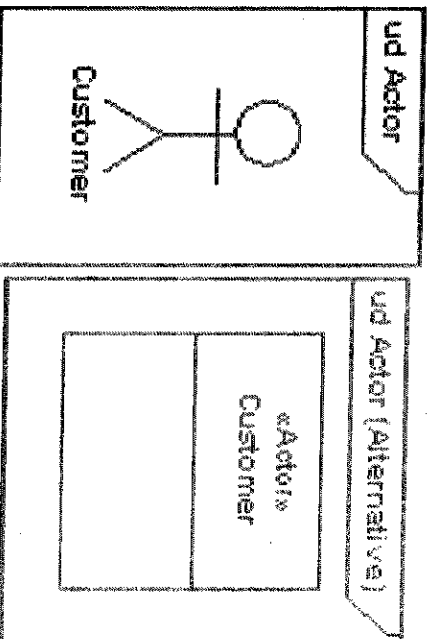
Partition : An activity partition is shown as either a horizontal or vertical swim lane. In the following diagram, the partitions are used to separate actions within an activity into those performed by the accounting department and those performed by the customer.



Use Case Model Actors



A use case diagram shows the interaction between the system and entities external to the system. These external entities are referred to as actors. Actors represent roles which may include human users, external hardware or other systems. An actor is usually drawn as a named stick figure, or alternatively as a class rectangle with the «actor» keyword.

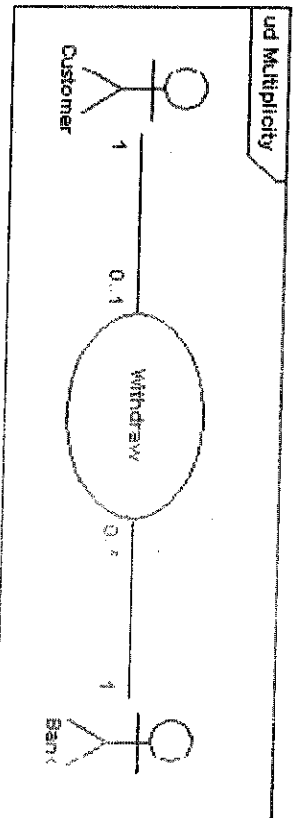


A use case typically Includes:

- Name and description
- Requirements
- Constraints
- Scenarios
- Scenario diagrams
- Additional information.

Name and Description

A use case is normally named as a verb-phrase and given a brief informal textual description

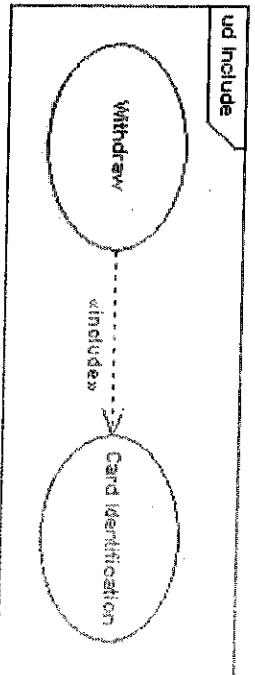


Requirements

The requirements define the formal functional requirements that a use case must supply to the end user. They correspond to the functional specifications found in structured methodologies. A requirement is a contract or promise that the use case will perform an action or provide some value to the system.

Including Use Cases

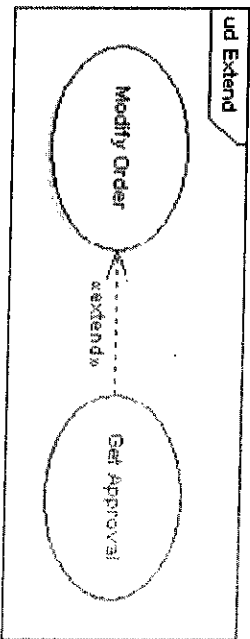
Use cases may contain the functionality of another use case as part of their normal processing. In general, it is assumed that any included use case will be called every time the basic path is run. An example of this is to have the execution of the use case <Card Identification> to be run as part of a use case <Withdraw>.



Use Cases may be included by one or more Use Case, helping to reduce the level of duplication of functionality by factoring out common behavior into Use Cases that are reused many times.

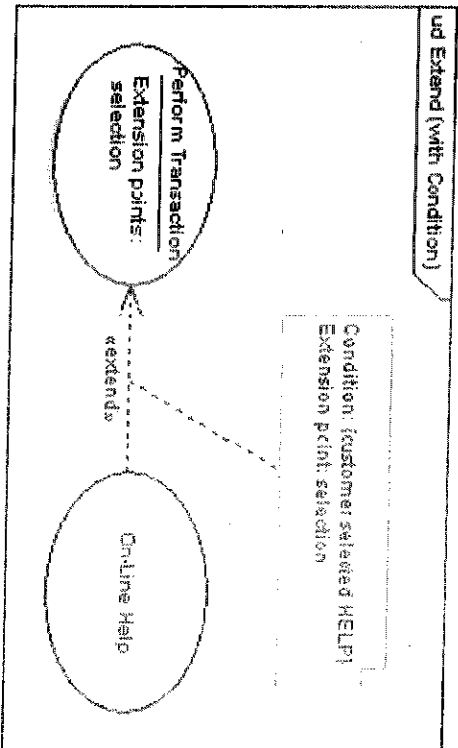
Extending Use Cases

One use case may be used to extend the behavior of another; this is typically used in exceptional circumstances. For example, if before modifying a particular type of customer order, a user must get approval from some higher authority, then the <Get Approval> use case may optionally extend the regular <Modify Order> use case.



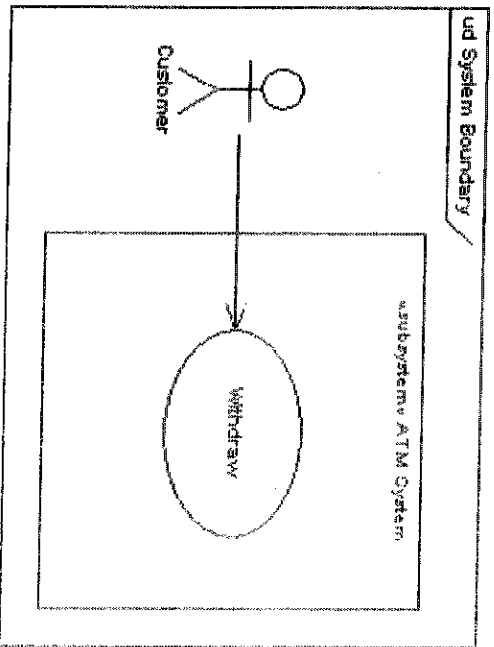
Extension Points

The point at which an extending use case is added can be defined by means of an extension point.



System Boundary

It is usual to display use cases as being inside the system and actors as being outside the system.

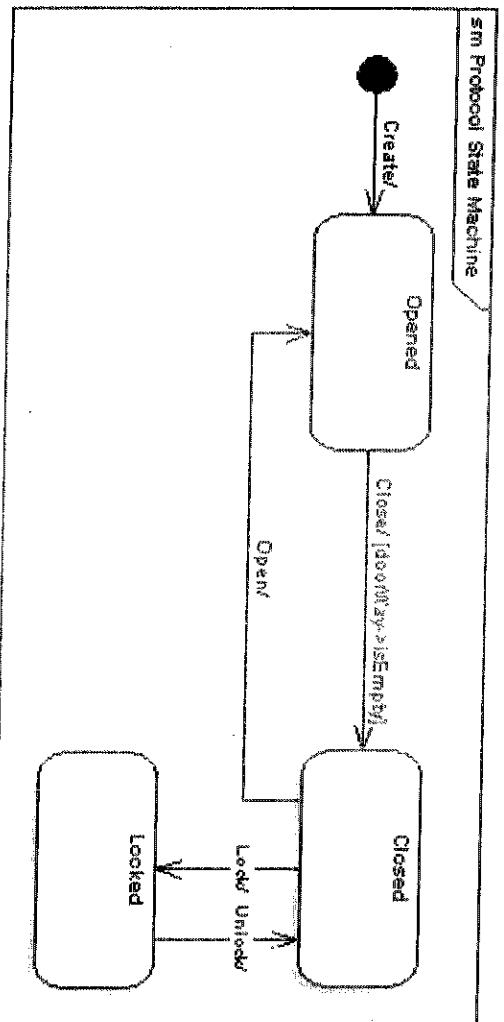


THE STATE MACHINE MODEL

A state machine diagram models the behaviour of a single object, specifying the sequence of events that an object goes through during its lifetime in response to events.

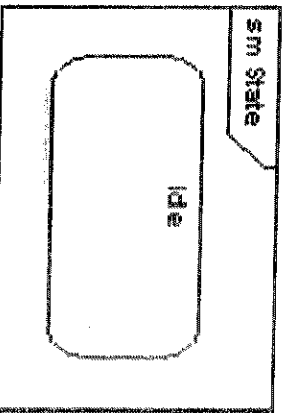
State diagram standardized notation to describe many systems, from computer programs to business processes.

As an example, the following state machine diagram shows the states that a door goes through during its lifetime.



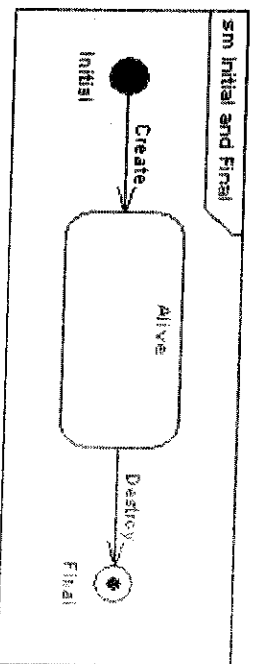
The door can be in one of three states: "Opened", "Closed" or "Locked". It can respond to the events Open, Close, Lock and Unlock. Notice that not all events are valid in all states; for example, if a door is opened, you cannot lock it until you close it. Also notice that a state transition can have a guard condition attached: if the door is Opened, it can only respond to the Close event if the condition doorway->isEmpty is fulfilled. The syntax and conventions used in state machine diagrams will be discussed in full in the following sections.

States : A state is denoted by a round-cornered rectangle with the name of the state written inside it.



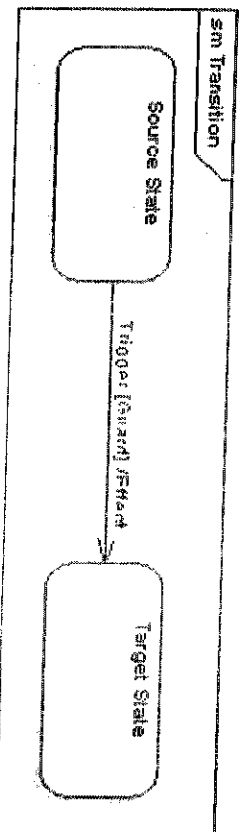
Initial and Final States

The initial state is denoted by a filled black circle and may be labelled with a name. The final state is denoted by a circle with a dot inside and may also be labelled with a name.



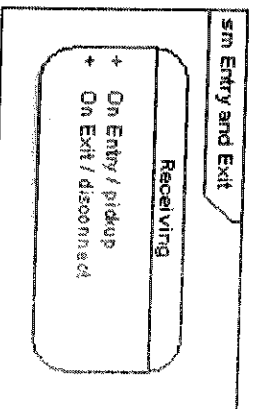
Transitions : Transitions from one state to the next are denoted by lines with arrowheads. A transition may have a trigger, a guard and an effect, as below.

"Trigger" is the cause of the transition, which could be a signal, an event, a change in some condition, or the passage of time. "Guard" is a condition which must be true in order for the trigger to cause the transition. "Effect" is an action which will be invoked directly on the object that owns the state machine as a result of the transition.



State Actions

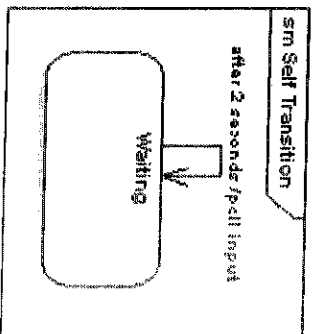
In the transition example above, an effect was associated with the transition. If the target state had many transitions arriving at it, and each transition had the same effect associated with it, it would be better to associate the effect with the target state rather than the transitions. This can be done by defining an entry action for the state. The diagram below shows a state with an entry action and an exit action.



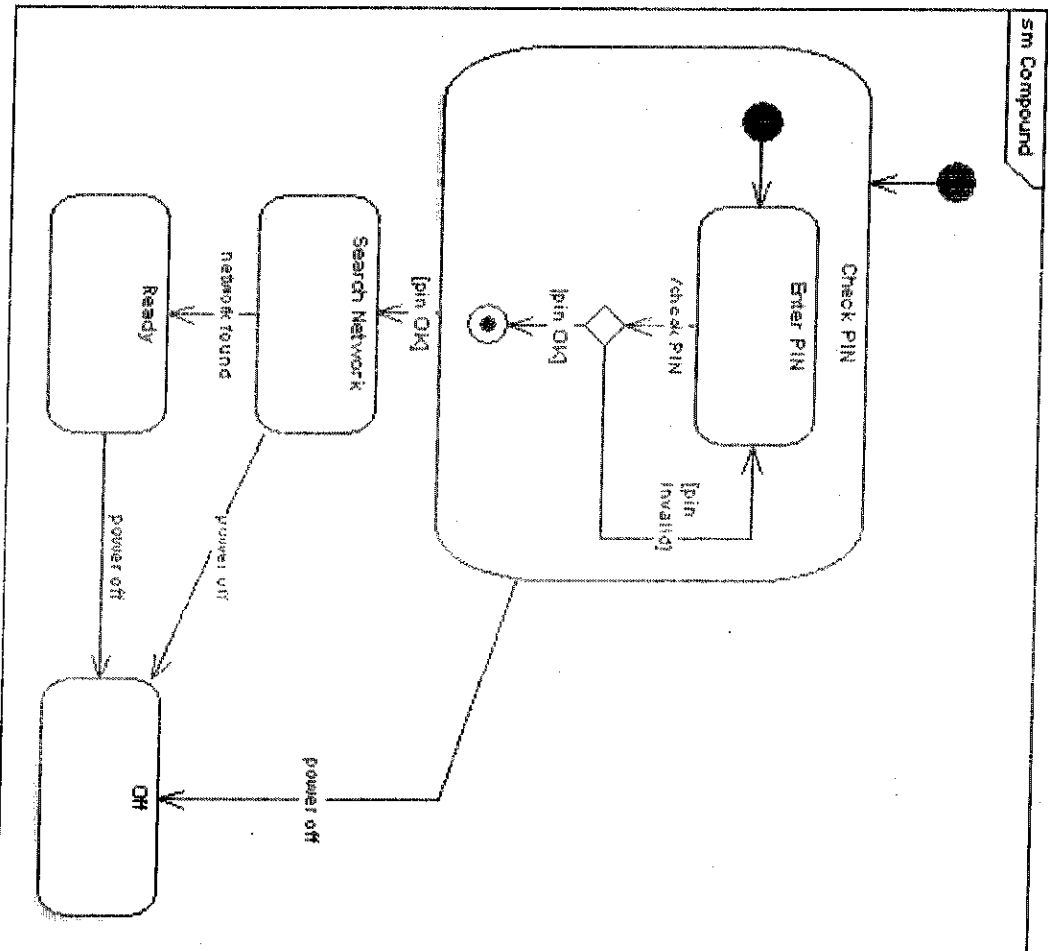
It is also possible to define actions that occur on events, or actions that always occur. It is possible to define any number of actions of each type.

Self-Transitions

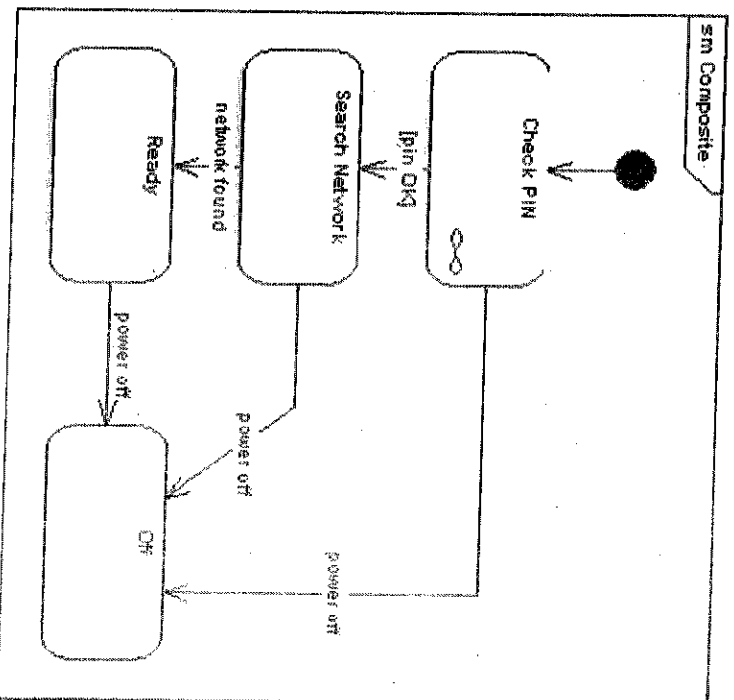
A state can have a transition that returns to itself, as in the following diagram. This is most useful when an effect is associated with the transition.



Compound States

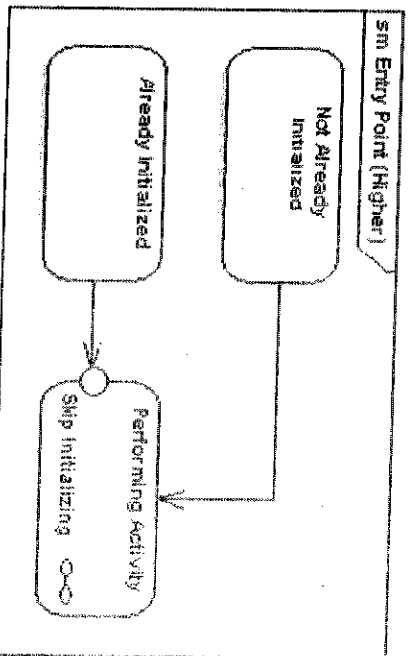


A state machine diagram may include sub-machine diagrams, as in the example below. The alternative way to show the same information is as follows.



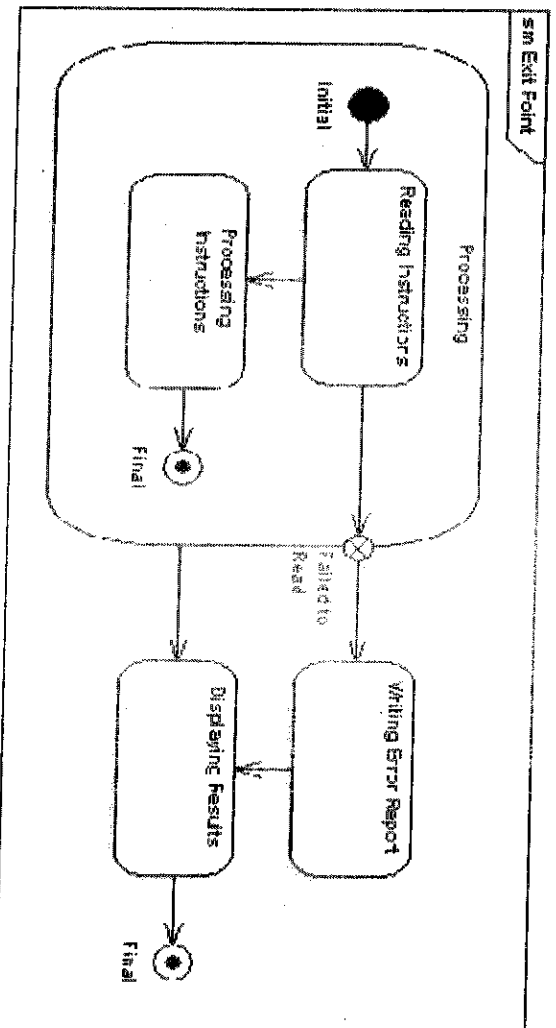
The notation in the above version indicates that the details of the Check PIN submachine are shown in a separate diagram.

Entry Point : Sometimes you won't want to enter a submachine at the normal initial state. For example, in the following sub-machine it would be normal to begin in the "Initializing" state, but if for some reason it wasn't necessary to perform the initialization, it would be possible to begin in the "Ready" state by transitioning to the named entry point.



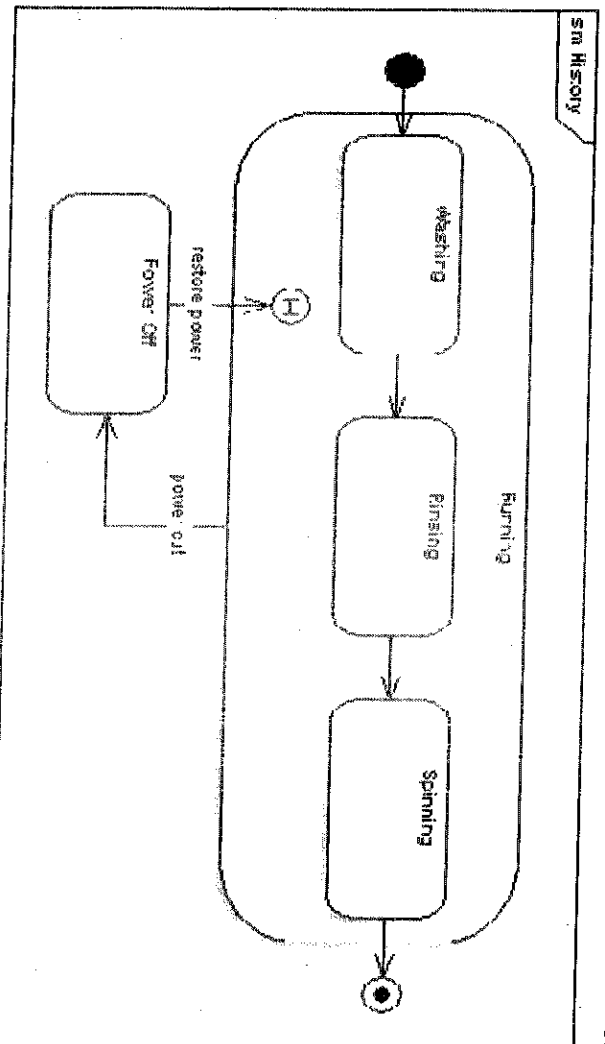
The following diagram shows the state machine one level up

Exit Point: In a similar manner to entry points, it is possible to have named alternative exit points. The following diagram gives an example where the state executed after the main processing state depends on which route is used to transition out of the state.

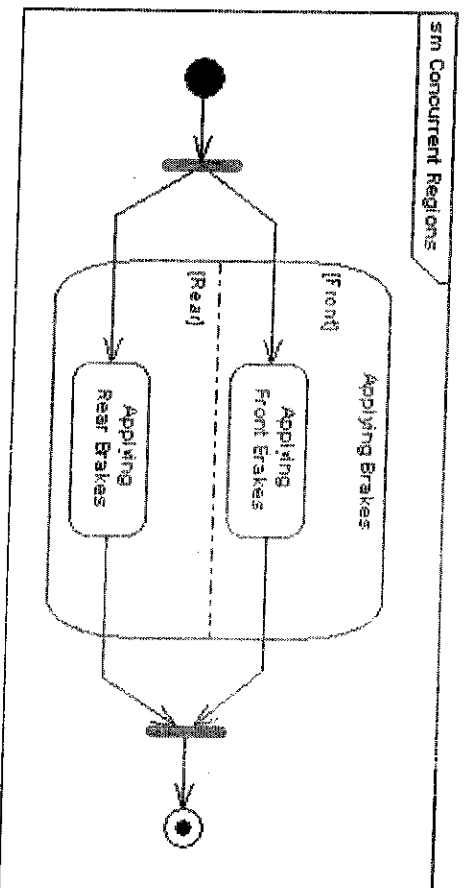


History States : A history state is used to remember the previous state of a state machine when it was interrupted. The following diagram illustrates the use of history states. The example is a state machine belonging to a washing machine.

In this state machine, when a washing machine is running, it will progress from "Washing" through "Rinsing" to "Spinning". If there is a power cut, the washing machine will stop running and will go to the "Power Off" state. Then when the power is restored, the Running state is entered at the "History State" symbol meaning that it should resume where it last left-off.



Concurrent Regions: A state may be divided into regions containing sub-states that exist and execute concurrently. The example below shows that within the state "Applying Brakes", the front and rear brakes will be operating simultaneously and independently. Notice the use of fork and join pseudo-states, rather than choice and merge pseudo-states. These symbols are used to synchronize the concurrent threads.



THE INTERACTION DIAGRAM

Interaction diagrams, a subset of behavior diagrams, emphasize the flow of control and data among the things in the system being modelled: It comprise of Communication, Interaction, Sequence and Timing diagrams.

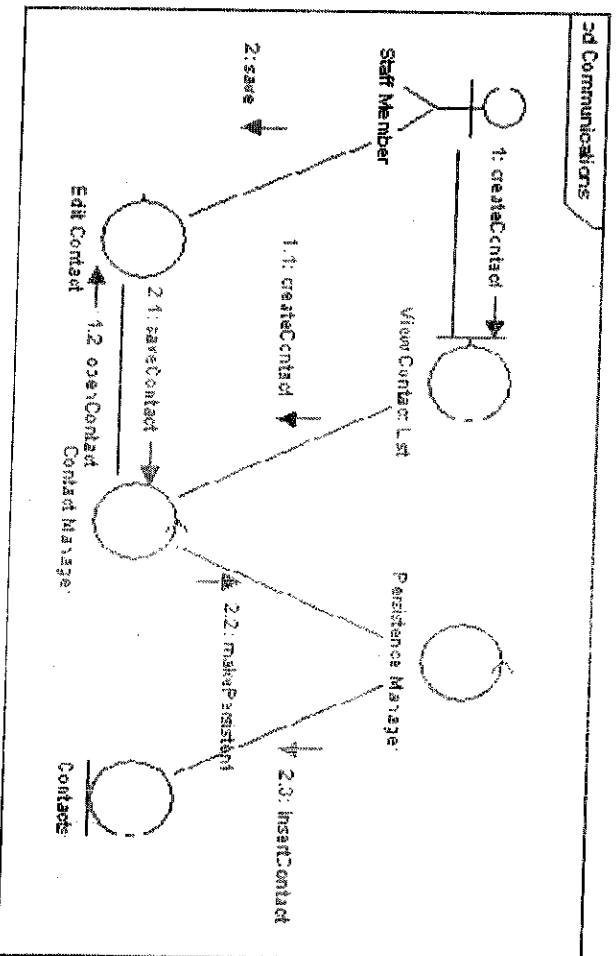
Communication Diagrams

A communication diagram, is an interaction diagram that shows similar information to sequence diagrams but its primary focus is on object relationships.

Communication diagram shows the interactions between objects or parts in terms of sequenced messages. They represent a combination of information taken from Class, Sequence, and Use Case Diagrams describing both the static structure and dynamic behavior of a system.

On communication diagrams, objects are shown with association connectors between them. Messages are added to the associations and show as short arrows pointing in the direction of the message flow. The sequence of messages is shown through a numbering scheme.

The following two diagrams show a communication diagram and the sequence diagram that shows the same information. Although it is possible to derive the sequencing of messages in the communication diagram from the numbering scheme, it isn't immediately visible. What the communication diagram does show quite clearly though, is the full set of messages passed between adjacent objects.

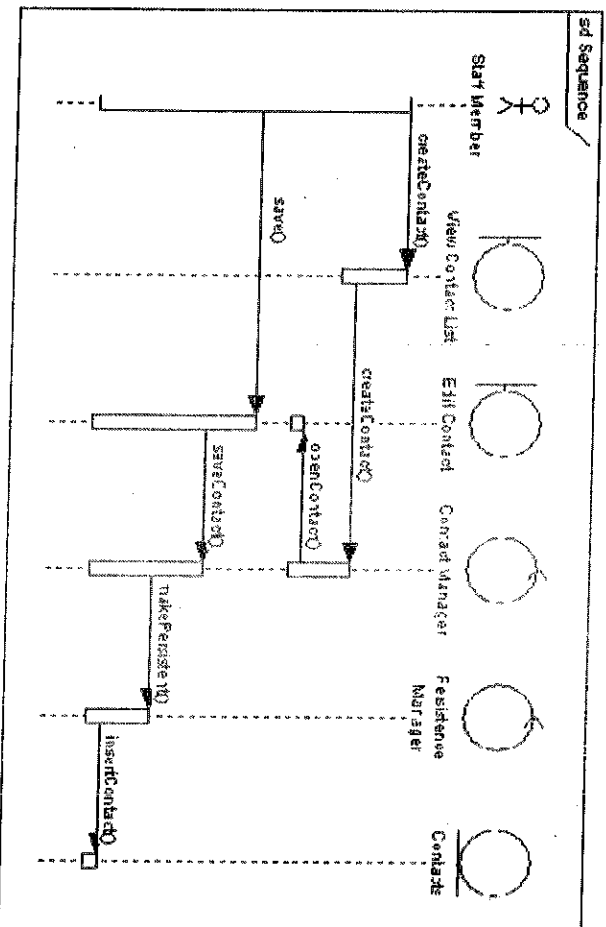


A sequence diagram is a form of interaction diagram which shows objects as lifelines running down the page, with their interactions over time represented as messages drawn as arrows from the source lifeline to the target lifeline.

The diagram's purpose

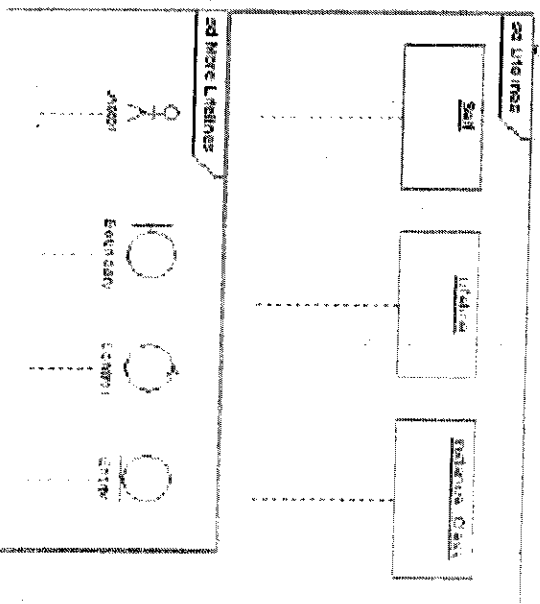
The sequence diagram is used primarily to show the interactions between objects in the sequential order that those interactions occur. Much like the class diagram, developers typically think sequence diagrams were meant exclusively for them. However, an organization's business staff can find sequence diagrams useful to communicate how the business currently works by showing how various business objects interact. Besides documenting an organization's current affairs, a business-level sequence diagram can be used as a requirements document to communicate requirements for a future system implementation. During the requirements phase of a project, analysts can take use cases to the next level by providing a more formal level of refinement. When that occurs, use cases are often refined into one or more sequence diagrams.

Sequence of messages. Also indicates the lifespans of objects relative to those messages. Sequence diagrams are good at showing which objects communicate with which other objects; and what messages trigger those communications. Sequence diagrams are not intended for showing complex procedural logic.

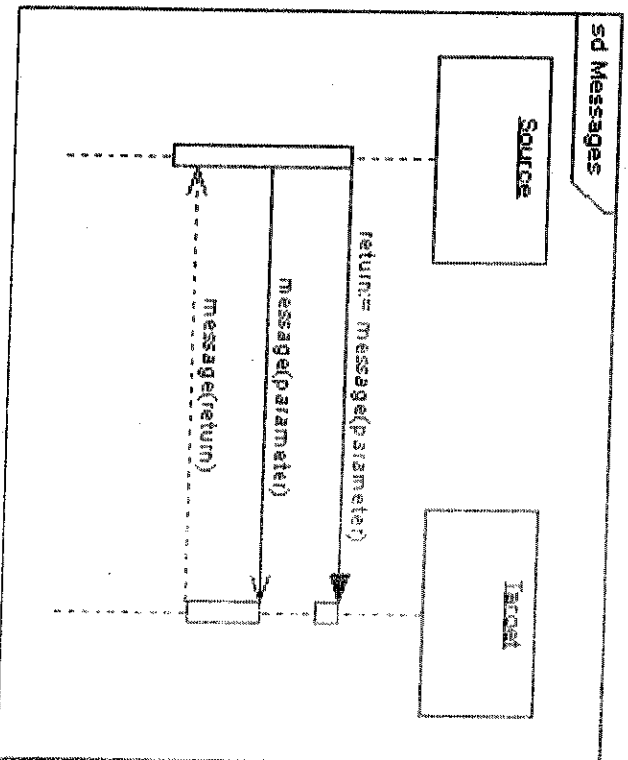


Lifelines : A lifeline represents an individual participant in a sequence diagram. A lifeline will usually have a rectangle containing its object name. If its name is "self", that indicates that the lifeline represents the classifier which owns the sequence diagram.

Sometimes a sequence diagram will have a lifeline with an actor element symbol at its head. This will usually be the case if the sequence diagram is owned by a use case. Boundary, control and entity elements from robustness diagrams can also own lifelines.



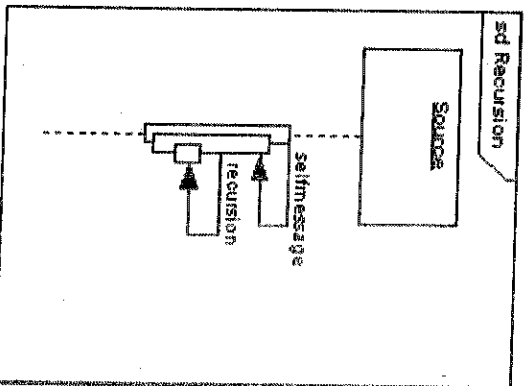
Messages : Messages are displayed as arrows. Messages can be complete, lost or found; synchronous or asynchronous; call or signal. In the following diagram, the first message is a synchronous message (denoted by the solid arrowhead) complete with an implicit return message; the second message is asynchronous (denoted by line arrowhead), and the third is the asynchronous return message (denoted by the dashed line).



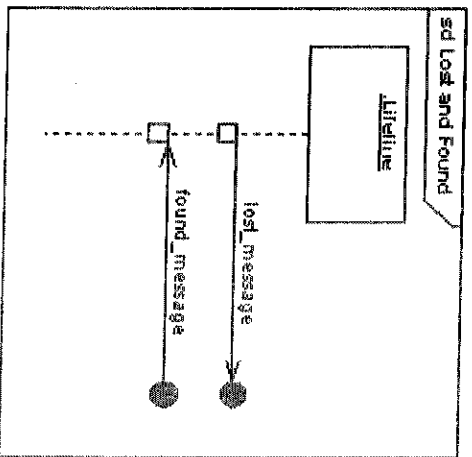
Execution Occurrence

A thin rectangle running down the lifeline denotes the execution occurrence, or activation of a focus of control. In the previous diagram, there are three execution occurrences. The first is the source object sending two messages and receiving two replies; the second is the target object receiving a synchronous message and returning a reply; and the third is the target object receiving an asynchronous message and returning a reply.

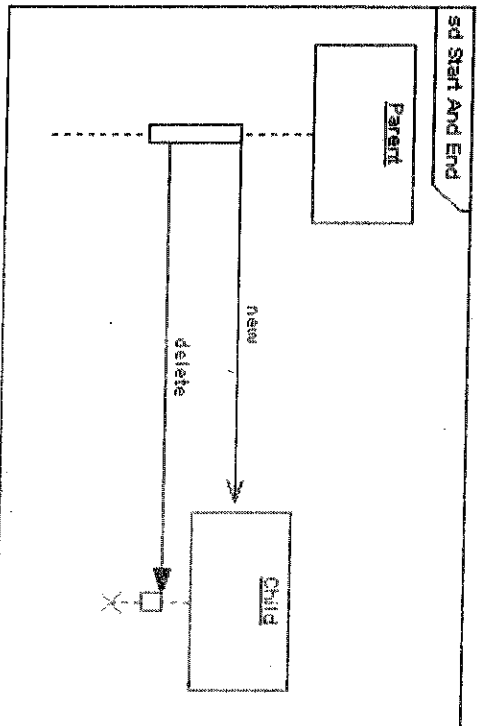
Self Message: A self message can represent a recursive call of an operation, or one method calling another method belonging to the same object. It is shown as creating a nested focus of control in the lifeline's execution occurrence.



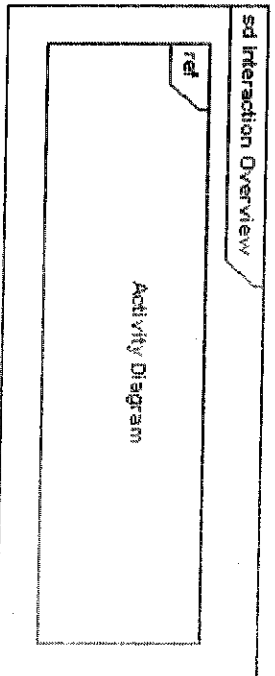
Lost and Found Messages: Lost messages are those that are either sent but do not arrive at the intended recipient, or which go to a recipient not shown on the current diagram. Found messages are those that arrive from an unknown sender, or from a sender not shown on the current diagram. They are denoted going to or coming from an endpoint element.



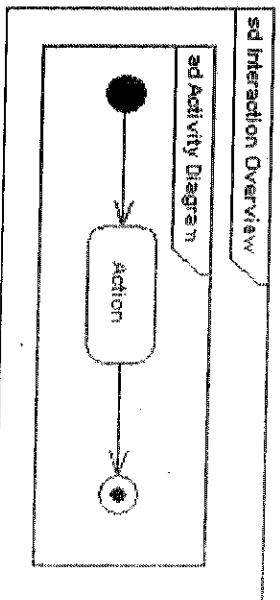
Lifetime Start and End : A lifetime may be created or destroyed during the timescale represented by a sequence diagram. In the latter case, the lifetime is terminated by a stop symbol, represented as a cross. In the former case, the symbol at the head of the lifetime is shown at a lower level down the page than the symbol of the object that caused the creation. The following diagram shows an object being created and destroyed.



Interaction Occurrence : Interaction occurrences are references to existing interaction diagrams. An interaction occurrence is shown as a reference frame; that is, a frame with "ref" in the top-left corner. The name of the diagram being referenced is shown in the center of the frame.



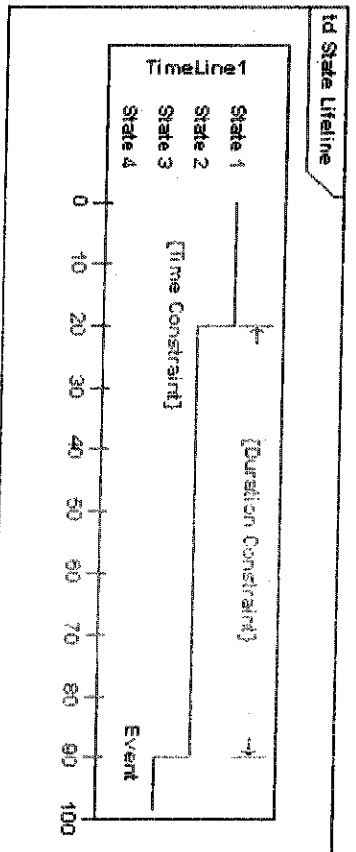
Interaction Element : Interaction elements is displayed as representation of existing interaction diagrams within a rectangular frame. They differ in that they display the contents of the references diagram inline.



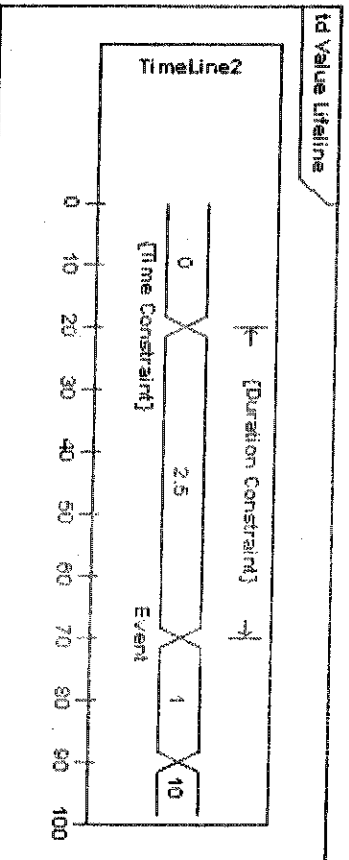
The Timing Diagram: These are a specific type of interaction diagram, where the focus is on timing constraints. UML timing diagrams are used to display the change in state or value of one or more elements over time. It can also show the interaction between timed events and the time and duration constraints that govern them.

State Lifeline

A state lifeline shows the change of state of an item over time. The X-axis displays elapsed time in whatever units are chosen, while the Y-axis is labelled with a given list of states. A state lifeline is shown below.



Value Lifeline: A value lifeline shows the change of value of an item over time. The X-axis displays elapsed time in whatever units are chosen, the same as for the state lifeline. The value is shown between the pair of horizontal lines which cross over at each change in value. A value lifeline is shown below.



CASE TOOL APPLICATION (Alova window)

